

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU & TRÍ TUỆ NHÂN
TẠO

ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG TẠO THUYẾT MINH VIDEO TỰ
ĐỘNG, SỬ DỤNG VISION-LANGUAGE MODEL (VLM) VÀ
ZERO-SHOT VOICE CLONING TIẾNG VIỆT

Người hướng dẫn: TS. TRẦN THẾ VŨ

Sinh viên thực hiện: NGÔ NGUYỄN TẤN QUÂN

Số thẻ sinh viên: 102220033

Lớp: 22T_KHDL

Đà Nẵng, 06/2026

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU & TRÍ TUỆ NHÂN
TẠO

ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG TẠO THUYẾT MINH VIDEO TỰ
ĐỘNG, SỬ DỤNG VISION-LANGUAGE MODEL (VLM) VÀ
ZERO-SHOT VOICE CLONING TIẾNG VIỆT

Người hướng dẫn: TS. TRẦN THẾ VŨ

Sinh viên thực hiện: NGÔ NGUYỄN TẤN QUÂN

Số thẻ sinh viên: 102220033

Lớp: 22T_KHDL

Đà Nẵng, 06/2026

[illegible]

[illegible]

TÓM TẮT

Tên đề tài: Xây dựng hệ thống tạo thuyết minh video tự động, sử dụng Vision-Language Model (VLM) và Zero-shot Voice Cloning tiếng Việt

Sinh viên thực hiện: Ngô Nguyễn Tân Quân

Số thẻ SV: 102220033 Lớp: 22T_KHDL

Nhu cầu việt hóa các nội dung video nước ngoài, đặc biệt là video hướng dẫn công nghệ, bài giảng trực tuyến và video demo, ngày càng tăng mạnh. Đề án này nghiên cứu và phát triển một hệ thống tự động hóa toàn bộ quy trình trích xuất phụ đề và tạo thuyết minh tiếng Việt cho các video tiếng Anh gốc chưa có thuyết minh giọng nói (chỉ có chữ hoặc phụ đề hiển thị) và bắt buộc phải có sẵn phụ đề tiếng Anh gắn sẵn (hard-sub). Hệ thống không xử lý hay dịch thuật từ phần âm thanh (audio) gốc của video.

Hệ thống kết hợp sức mạnh của mô hình ngôn ngữ - thị giác (Vision-Language Model - VLM) để hiểu ngữ cảnh, mô hình nhận dạng ký tự quang học chuyên dụng cho video (GLM-OCR) để trích xuất phụ đề cứng, và mô hình nhân bản giọng nói zero-shot (VieNeu-TTS v2 Turbo) để tổng hợp tiếng nói tự nhiên bằng tiếng Việt giữ nguyên màu giọng của người nói tham chiếu mà không cần huấn luyện lại.

Để giải quyết vấn đề lệch thời gian giữa tiếng Anh và tiếng Việt (tiếng Việt thường dài hơn tiếng Anh 30%), đề án đề xuất và triển khai thuật toán đồng bộ âm thanh thông minh kết hợp co giãn thời gian có ngân sách (Budgeted Time-Stretching) dựa trên công cụ Rubberband và kỹ thuật dịch chuyển tích lũy (Slip Cascade). Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, đạt độ tự nhiên của giọng nói thuyết minh cao và khả năng đồng bộ thời gian dưới ngưỡng sai số cho phép, sẵn sàng ứng dụng trong việc tự động hóa sản xuất thuyết minh cho nội dung số.

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ tên sinh viên: Ngô Nguyễn Tấn Quân

Số thẻ sinh viên: 102220033

Lớp: 22T_KHDL Khoa: CNTT

Ngành: Khoa học dữ liệu & Trí tuệ nhân tạo

1. Tên đề tài đồ án:

Xây dựng hệ thống tạo thuyết minh video tự động, sử dụng Vision-Language Model (VLM) và Zero-shot Voice Cloning tiếng Việt.

2. Đề tài thuộc diện: ☐ Có ký kết thỏa thuận sở hữu trí tuệ đối với kết quả thực hiện

3. Các số liệu và dữ liệu ban đầu:

- Video hướng dẫn/demo công nghệ tiếng Anh có phụ đề tiếng Anh gắn sẵn (hard-sub)
- Tập âm thanh tham chiếu (giọng mẫu thuyết minh) dài 3–10 giây để nhân bản giọng.
- Các mô hình pre-trained self-hosted: VLM (API Gemini 2.5 Flash Lite và Qwen3.5-2B chạy local), GLM-OCR (nhận dạng phụ đề), VieNeu-TTS v2 Turbo (nhân bản giọng zero-shot); thư viện Rubberband, Pydub, FFmpeg, Librosa, Resemblyzer.
- Cấu hình thực nghiệm: CPU Intel Core i7-13700HX, GPU NVIDIA RTX 4060 Laptop (8 GB VRAM), 16 GB RAM, Windows 11; Python 3.11, CUDA 12.1, PyTorch 2.2.

4. Nội dung các phần thuyết minh và tính toán:

- Mở đầu: lý do chọn đề tài, mục tiêu, đối tượng & phạm vi, phương pháp nghiên cứu.
- Chương 1 – Tổng quan: khái niệm thuyết minh video tự động và các thách thức kỹ thuật (chênh lệch độ dài Việt–Anh, nhiều văn bản màn hình, ảo giác thời gian của VLM).
- Chương 2 – Cơ sở lý thuyết: kiến trúc Transformer & Self-Attention, mô hình VLM (Gemini, Qwen-VL), nguyên lý OCR & GLM-OCR, nền tảng Neural TTS & zero-shot voice cloning (VieNeu-TTS), Phase Vocoder, và các độ đo đánh giá.
- Chương 3 – Phân tích & thiết kế: kiến trúc 3 mô-đun và pipeline xử lý 8 bước; thuật toán trích xuất & nhóm phân đoạn phụ đề (SequenceMatcher), bộ phân loại lời thoại bằng VLM, thuật toán co giãn có ngân sách (Budgeted Time-Stretching)

và trượt tích lũy (Slip Cascade); phân tích yêu cầu, sơ đồ Use-case/Sequence/Component, thiết kế API và giao diện Web (FastAPI + Vite React).

- Chương 4 – Thực nghiệm & đánh giá: môi trường triển khai, kịch bản kiểm thử chức năng, đánh giá chất lượng (rubric Adequacy/Fluency cho dịch thuật; chỉ số khách quan Speaker Similarity, độ trễ đồng bộ, Caption Drift), phân tích lỗi, so sánh VLM-mode và OCR-mode.
- Kết luận & hướng phát triển.
- Phân tích toán: công thức độ tương đồng chuỗi $R = 2M/(L_1+L_2)$; thuật toán co giãn thời lượng với ngưỡng nén $\leq 1,25x$ và cơ chế Slip Cascade; tính độ tương đồng giọng bằng cosine similarity; đo độ trễ đồng bộ (Mean/p95 Delay) và Caption Drift.

5. Các bản vẽ, đồ thị (ghi rõ các loại và kích thước bản vẽ):

- Sơ đồ luồng dữ liệu pipeline xử lý 8 bước; sơ đồ kiến trúc khối Transformer & Self-Attention; sơ đồ kiến trúc VLM, Qwen-VL, GLM-OCR, Zero-shot Voice Cloning (VieNeu-TTS); pipeline Neural TTS.
- Sơ đồ Use-case, sơ đồ tuần tự (Sequence), sơ đồ thành phần (Component) của hệ thống.
- Lưu đồ thuật toán nhóm phân đoạn phụ đề; sơ đồ minh họa thuật toán Budgeted Time-Stretching + Slip Cascade.
- Biểu đồ độ tương đồng giọng nói (Speaker Similarity) qua các giai đoạn; ảnh chụp giao diện Web Dashboard kết quả.

6. Họ tên người hướng dẫn: TS. Trần Thế Vũ

7. Ngày giao nhiệm vụ đồ án: 19/03/2026.

8. Ngày hoàn thành đồ án: 17/06/2026.

Đà Nẵng, ngày tháng năm 2026

Trưởng Bộ môn

Người hướng dẫn

LỜI NÓI ĐẦU VÀ CẢM ƠN

Trong những năm gần đây, sự phát triển vượt bậc của Trí tuệ Nhân tạo (AI) đa phương thức đã mở ra nhiều hướng ứng dụng mới trong việc xử lý phương tiện truyền thông. Đề tài “Xây dựng hệ thống tạo thuyết minh video tự động, sử dụng Vision-Language Model (VLM) và Zero-shot Voice Cloning tiếng Việt” được thực hiện nhằm giải quyết bài toán thực tế: tự động tạo thuyết minh tiếng Việt cho các video gốc (chỉ có phụ đề tiếng Anh và không có sẵn giọng thuyết minh), giúp người xem Việt Nam dễ dàng tiếp cận tri thức từ các nguồn tài liệu quốc tế.

Em xin chân thành cảm ơn các thầy cô giáo khoa Công nghệ Thông tin - Trường Đại học Bách khoa, Đại học Đà Nẵng đã truyền đạt những kiến thức quý báu và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập và thực hiện đồ án. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến Giảng viên hướng dẫn TS. Trần Thế Vũ đã tận tình chỉ bảo, định hướng và đưa ra những góp ý chuyên môn quan trọng giúp em hoàn thiện đồ án này.

CAM ĐOAN

Em tên là: Ngô Nguyễn Tấn Quân

Mã số sinh viên: 102220033

Lớp: 22T_KHDL

Tôi xin cam đoan đồ án tốt nghiệp này là công trình nghiên cứu do chính tôi thực hiện dưới sự hướng dẫn của Giảng viên hướng dẫn. Các số liệu, kết quả thực nghiệm nêu trong báo cáo là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác. Mọi tài liệu tham khảo, mã nguồn kế thừa hoặc sử dụng từ các thư viện mở đều được trích dẫn và liệt kê đầy đủ trong danh mục tài liệu tham khảo theo đúng quy định về liêm chính học thuật của Trường Đại học Bách khoa - Đại học Đà Nẵng.

Sinh viên thực hiện

Ngô Nguyễn Tấn Quân

MỤC LỤC

TÓM TẮT	
NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP	
LỜI NÓI ĐẦU VÀ CẢM ƠN	i
CAM ĐOAN.....	ii
DANH SÁCH HÌNH ẢNH.....	vi
DANH SÁCH BẢNG.....	vii
DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT	viii
MỞ ĐẦU.....	1
1. Lý do chọn đề tài.....	1
2. Mục tiêu đề tài.....	1
3. Đối tượng và phạm vi nghiên cứu	2
4. Phương pháp nghiên cứu	2
CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG TẠO THUYẾT MINH VIDEO TỰ ĐỘNG	4
1.1. Khái niệm thuyết minh tự động (AI Video Narration).....	4
1.2. Thách thức kỹ thuật của bài toán thuyết minh Việt hóa	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG.....	6
2.1. Kiến trúc Transformer và cơ chế Self-Attention.....	6
2.2. Kiến trúc Vision-Language Model (VLM).....	7
2.3. Nhận dạng ký tự quang học và mô hình GLM-OCR.....	9
2.4. Công nghệ nhân bản giọng nói Zero-shot VieNeu-TTS v2 Turbo.....	11
2.5. Thư viện đồng bộ và xử lý âm thanh Rubberband & Pydub.....	13
2.6. Biểu diễn và đo lường đặc trưng giọng nói (Speaker Embedding).....	14
2.7. Các độ đo đánh giá chất lượng	14
2.8. Lựa chọn công nghệ cho hệ thống và so sánh phương án.....	15

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	16
3.1. Kiến trúc tổng thể và luồng dữ liệu (Data Pipeline)	16
3.2. Thiết kế Module Trích xuất và Dịch thuật (Module 1 - VLM/OCR)	17
3.2.1. Quyết định kỹ thuật: Từ bỏ VLM-mode chuyển sang OCR-mode	17
3.2.2. Trích xuất Timeline Phụ đề bằng CaptionTimeline	18
3.2.3. Lọc lời thoại bằng VLM Narration Classifier	19
3.2.4. Ghép nối Phụ đề ngắn và Kéo dài thời gian kết thúc	20
3.2.5. Dịch thuật ngữ cảnh VLM từng phân đoạn	21
3.3. Thiết kế Module Tổng hợp Giọng nói (Module 2 - TTS)	21
3.4. Thiết kế Module Đồng bộ Âm thanh (Module 3 - Sync/Render)	21
3.4.1. Thuật toán Co giãn có Ngân sách (Budgeted Time-Stretching)	21
3.4.2. Cơ chế Trượt tích lũy (Slip Cascade)	22
3.4.3. Trộn âm thanh và Render video bằng FFmpeg	23
3.5. Thiết kế Hệ thống Web UI (FastAPI + Vite React)	23
3.6. Phân tích yêu cầu hệ thống	24
3.7. Mô hình hóa hệ thống	25
3.8. Thiết kế API và cấu trúc dữ liệu	26
3.9. Thiết kế Prompt và lưu đồ thuật toán	27
CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ	29
4.1. Môi trường triển khai thực nghiệm	29
4.2. Phương pháp Đánh giá Tích hợp Cấp Hệ thống (System-level Integration Evaluation)	29
4.3. Các chỉ số đo lường chi tiết	29
4.3.1. Đánh giá chất lượng dịch thuật	29
4.3.2. Đánh giá chất lượng nhân bản giọng nói	30
4.3.3. Đánh giá độ chính xác đồng bộ thời gian (Sync Accuracy)	30
4.3.4. Đánh giá độ lệch phụ đề OCR (Caption Drift)	30
4.4. Phân tích kết quả thực nghiệm	30

4.4.1. Kết quả Dịch thuật & Trích xuất OCR.....	31
4.4.2. Kết quả Nhân bản giọng nói (Speaker Similarity)	32
4.4.3. Kết quả Đồng bộ thời gian (Sync Accuracy) & Caption Drift.....	32
4.5. Kịch bản và quy trình kiểm thử chức năng	33
4.6. Đánh giá định tính chất lượng đầu ra	33
4.7. Phân tích lỗi và hướng khắc phục.....	34
4.8. So sánh định tính giữa VLM-mode và OCR-mode	34
4.9. Nhận xét về thời gian xử lý	35
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	36
1. Kết luận chung	36
2. Những kết quả đạt được	36
3. Những đóng góp chính	36
4. Hạn chế của hệ thống	37
5. Hướng phát triển tiếp theo	37
TÀI LIỆU THAM KHẢO.....	38

DANH SÁCH HÌNH ẢNH

Hình 2.1: Sơ đồ kiến trúc khối Transformer _[12]	7
Hình 2.2: Cơ chế Self-attention và Multi-Head _[12]	7
Hình 2.3: Kiến trúc tổng quát của Vision-Language Model (VLM) _[10]	8
Hình 2.4: Kiến trúc dòng mô hình Qwen-VL (Qwen3.5-2B, chạy local) _[7]	9
Hình 2.5: Kiến trúc mô hình GLM-OCR _[13]	11
Hình 2.6: Pipeline Neural TTS _[15]	11
Hình 2.7: Cơ chế zero-shot voice-cloning theo XTTS _[1]	12
Hình 2.8: Kiến trúc Zero-shot Voice Cloning (VieNeu-TTS v2) được vẽ lại	13
Hình 3.1: Sơ đồ luồng dữ liệu pipeline 8 bước ở chế độ OCR	17
Hình 3.2: Minh họa thuật toán nhóm phân đoạn phụ đề bằng SequenceMatcher	19
Hình 3.3: Ví dụ bộ phân loại VLM Narration Classifier (narration / screen)	20
Hình 3.4: Minh họa thuật toán Co giãn có Ngân sách và cơ chế Slip Cascade	23
Hình 3.5: Giao diện Web Dashboard (FastAPI + Vite React)	24
Hình 3.6: Biểu đồ Use-case của hệ thống	25
Hình 3.7: Biểu đồ tuần tự xử lý một job thuyết minh	26
Hình 3.8: Sơ đồ thành phần hệ thống	26
Hình 3.9: Lưu đồ thuật toán trích xuất và nhóm phân đoạn phụ đề	28
Hình 4.1: Giao diện tiến trình chạy thực nghiệm	31
Hình 4.2: Giao diện tiến trình xử lý hoàn thành	31
Hình 4.3: Biểu đồ độ tương đồng giọng nói qua các giai đoạn xử lý	32

DANH SÁCH BẢNG

Bảng 2.1: Bảng so sánh các lựa chọn công nghệ chính của hệ thống	15
Bảng 3.1: Danh sách yêu cầu chức năng của hệ thống	25
Bảng 3.2: Danh sách yêu cầu phi chức năng của hệ thống	25
Bảng 3.3: Danh sách các API chính của hệ thống (RESTful).....	27
Bảng 4.1: Kết quả độ tương đồng giọng nói qua các giai đoạn xử lý	32
Bảng 4.2: Các kịch bản kiểm thử chức năng của hệ thống	33
Bảng 4.3: So sánh định tính giữa VLM-mode và OCR-mode	35

DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT

Viết tắt	Từ gốc tiếng Anh	Diễn giải tiếng Việt
VLM	Vision-Language Model	Mô hình Ngôn ngữ - Thị giác
TTS	Text-to-Speech	Tổng hợp giọng nói từ văn bản
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
SRT	SubRip Subtitle file	Định dạng tệp phụ đề tiêu chuẩn
FPS	Frames Per Second	Số khung hình trên giây
API	Application Programming Interface	Giao diện lập trình ứng dụng
BLEU	Bilingual Evaluation Understudy	Chỉ số đánh giá dịch thuật song ngữ tự động
RTF	Real-Time Factor	Hệ số thời gian thực (tỷ lệ thời gian xử lý trên thời lượng audio)
UI/UX	User Interface / User Experience	Giao diện người dùng / Trải nghiệm người dùng
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MLP	Multi-Layer Perceptron	Mạng nơ-ron perceptron nhiều lớp
CLIP	Contrastive Language-Image Pre-training	Mô hình tiền huấn luyện đối sánh ảnh - văn bản
SigLIP	Sigmoid Loss for Language-Image Pre-training	Bộ mã hóa thị giác dùng hàm mất mát sigmoid
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
VRAM	Video Random Access Memory	Bộ nhớ truy cập ngẫu nhiên của card đồ họa
SOTA	State-of-the-art	Kết quả tốt nhất hiện tại của lĩnh vực
MOS	Mean Opinion Score	Điểm đánh giá chủ quan trung bình về chất lượng giọng nói
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
CLI	Command-Line Interface	Giao diện dòng lệnh

MỞ ĐẦU

1. Lý do chọn đề tài

Sự bùng nổ của các nền tảng chia sẻ video trực tuyến đã tạo ra một kho tài nguyên học liệu khổng lồ (như YouTube, Udemy, Coursera). Tuy nhiên, rào cản ngôn ngữ là thách thức lớn nhất đối với người dùng Việt Nam khi tiếp cận các video hướng dẫn (tutorial) công nghệ, bài giảng và demo phần mềm bằng tiếng Anh. Nhiều video hướng dẫn công nghệ chỉ có thao tác quay màn hình hoặc chỉ hiển thị phụ đề tiếng Anh mà hoàn toàn không có giọng nói thuyết minh gốc đi kèm.

Quy trình tạo thuyết minh Việt hóa truyền thống yêu cầu người dịch dịch thuật thủ công, biên tập viên căn chỉnh thời gian phụ đề, thuyết minh viên thu âm độc lập, và kỹ thuật viên âm thanh dựng hậu kỳ ghép nối. Quy trình này đòi hỏi chi phí nhân sự lớn và thời gian sản xuất kéo dài.

Do đó, việc xây dựng một hệ thống tự động hóa hoàn toàn quy trình này (End-to-End Video Narration Generation) là vô cùng cần thiết. Đề tài hướng tới việc kết hợp các công nghệ AI tiên tiến bao gồm mô hình ngôn ngữ - thị giác (VLM), nhận dạng chữ viết chuyên dụng (OCR) và nhân bản giọng nói tức thời (Zero-shot Voice Cloning) để tự động tạo thuyết minh tiếng Việt chất lượng cao cho các video gốc bắt buộc có sẵn phụ đề tiếng Anh, giúp tiết kiệm tối đa nguồn lực hậu kỳ.

2. Mục tiêu đề tài

Mục tiêu tổng quát: Xây dựng hệ thống phần mềm tạo thuyết minh tiếng Việt tự động cho các video hướng dẫn tiếng Anh bằng cách kết hợp trích xuất chữ viết phụ đề trên màn hình, dịch thuật ngữ cảnh, nhân bản giọng thuyết minh từ một tệp âm thanh tham chiếu ngắn (3-10 giây) và tự động đồng bộ hóa âm thanh tiếng Việt khớp với hình ảnh video gốc.

Mục tiêu cụ thể:

- Trích xuất chính xác dòng phụ đề tiếng Anh và thời gian xuất hiện từ video (đối với video bắt buộc có phụ đề tiếng Anh sẵn) bằng OCR.
- Lọc bỏ các chữ viết tĩnh trên giao diện màn hình (UI chrome, thông tin bản quyền) để chỉ giữ lại lời thoại tương thuật thực tế nhờ mô hình VLM phân loại.

3. Dịch thuật ngữ nghĩa chính xác sang tiếng Việt dựa trên việc đọc hiểu ngữ cảnh hình ảnh của VLM.
4. Nhân bản giọng thuyết minh bằng tiếng Việt với độ tương đồng giọng gốc cao (Speaker Similarity ≥ 0.75) sử dụng mô hình zero-shot TTS.
5. Đồng bộ hóa thời gian phát âm thanh thuyết minh tiếng Việt khớp với thời gian hiển thị phụ đề trên video gốc (sai lệch ≤ 0.5 giây) nhờ thuật toán co giãn thời gian giữ nguyên cao độ.
6. Xây dựng ứng dụng web hoàn chỉnh có giao diện Dashboard tương tác hiển thị tiến trình thời gian thực.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Mô hình ngôn ngữ - thị giác: API Gemini 2.5 Flash Lite và mô hình chạy local Qwen3.5-2B.
- Mô hình nhận dạng chữ viết trên video: GLM-OCR.
- Mô hình nhân bản giọng nói zero-shot tiếng Việt: VieNeu-TTS v2 Turbo.
- Các kỹ thuật xử lý tín hiệu số và đồng bộ đa phương tiện: thư viện Rubberband CLI và Pydub.

Phạm vi nghiên cứu: Đề án tập trung nghiên cứu phương pháp thiết kế prompt, tối ưu hóa pipeline tích hợp các mô hình pre-trained có sẵn (self-hosted) và phát triển thuật toán căn chỉnh đồng bộ thời lượng âm thanh thuyết minh. Đề án không tập trung vào việc huấn luyện lại (fine-tune) hoặc xây dựng các mô hình nền tảng từ đầu. Hệ thống chấp nhận đầu vào là các video dạng tutorial, bài giảng trực tuyến bắt buộc phải có sẵn phụ đề tiếng Anh gắn sẵn (hard-sub) và không xử lý phần âm thanh thuyết minh gốc.

4. Phương pháp nghiên cứu

Phương pháp lý thuyết: Nghiên cứu các kiến trúc VLM đa phương thức, cơ chế attention trong xử lý văn bản - hình ảnh, kiến trúc mạng neural tổng hợp tiếng nói dạng zero-shot, và các mô hình biểu diễn đặc trưng giọng nói (Speaker Embeddings).

Phương pháp thực nghiệm: Lập trình xây dựng toàn bộ pipeline bằng ngôn ngữ Python, triển khai self-hosted các mô hình trên GPU cá nhân, chạy thử nghiệm hệ thống end-to-end trên các tệp video mẫu và đánh giá chất lượng sản phẩm đầu ra (bản dịch

phụ đề, file âm thanh thuyết minh, video đã lồng thuyết minh hoàn chỉnh) dựa trên việc đối chiếu kết quả dịch thuật và kiểm tra thủ công tính đồng bộ âm thanh - hình ảnh.

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG TẠO THUYẾT MINH VIDEO TỰ ĐỘNG

1.1. Khái niệm thuyết minh tự động (AI Video Narration)

Thuyết minh tự động là quá trình trích xuất nội dung từ các dòng chữ/phụ đề hiển thị trên video gốc, dịch thuật và tổng hợp giọng nói của ngôn ngữ đích, sau đó ghép nối trở lại vào video gốc sao cho khớp thời gian hiển thị của phân cảnh hình ảnh tương ứng. Quy trình này đòi hỏi sự kết hợp chặt chẽ của ba lĩnh vực chính trong AI:

Computer Vision (Thị giác máy tính): Nhận dạng các sự kiện, phát hiện chuyển cảnh, trích xuất khung hình và đọc văn bản phụ đề tiếng Anh gắn sẵn trên video bằng OCR.

Natural Language Processing (Xử lý ngôn ngữ tự nhiên): Dịch thuật ngữ nghĩa dựa trên văn cảnh và hình ảnh thực tế của video, đảm bảo thuật ngữ chuyên ngành được chuyển ngữ chính xác.

Speech Synthesis (Tổng hợp tiếng nói): Sinh giọng nói tự nhiên của ngôn ngữ đích, kết hợp nhân bản giọng nói (Voice Cloning) để tái tạo sắc thái giọng nói thuyết minh từ tệp âm thanh tham chiếu mẫu.

1.2. Thách thức kỹ thuật của bài toán thuyết minh Việt hóa

Độ chênh lệch độ dài ngôn ngữ: Tiếng Việt khi diễn đạt cùng một ý nghĩa kỹ thuật thường dài hơn tiếng Anh khoảng 30% về số ký tự và thời gian phát âm tự nhiên. Nếu không có giãn âm thanh thuyết minh, lời thoại tiếng Việt sẽ tràn sang phân cảnh tiếp theo, gây mất đồng bộ nghiêm trọng.

Hiện tượng dồn chữ và lệch pha: Khi tổng hợp tiếng nói từ văn bản đơn thuần, tốc độ nói tự nhiên của TTS có thể nhanh hoặc chậm hơn thời lượng của phân cảnh hình ảnh tương ứng.

Nhiều văn bản từ màn hình: Khi trích xuất chữ viết trên video hướng dẫn lập trình hoặc demo phần mềm, các mô hình OCR sẽ đọc tất cả các văn bản tĩnh xuất hiện trên màn hình (như tên nút bấm, mã nguồn, tiêu đề slide, thông tin bản quyền ở chân trang). Việc thuyết minh cho các văn bản tĩnh này là không chính xác vì chúng không phải là lời nói thoại thực sự.

Tính phi tuyến tính của VLM trong việc sinh thời gian: Ban đầu, đề tài đã thử nghiệm thiết kế pipeline dựa hoàn toàn vào VLM để trực tiếp trích xuất lời thoại và sinh mốc thời gian (VLM Mode). Tuy nhiên, thực nghiệm cho thấy các mô hình VLM rất dễ gặp hiện tượng ảo giác (hallucination), sinh ra các mốc thời gian tùy ý không khớp với

chuyển động thực tế hoặc dồn tất cả phụ đề dịch được vào một phần nhỏ của video. Từ hạn chế lớn này, đề án đã chuyển đổi thiết kế sang hướng tiếp cận lấy OCR làm trọng tâm để tạo timeline cơ sở vững chính xác (OCR Mode) cho các video có phụ đề tiếng Anh sẵn, sau đó dùng VLM làm bộ lọc và dịch thuật.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG

2.1. Kiến trúc Transformer và cơ chế Self-Attention

Transformer là kiến trúc mạng nơ-ron nền tảng cho hầu hết các mô hình ngôn ngữ lớn (LLM), mô hình ngôn ngữ - thị giác (VLM) và nhiều mô hình tổng hợp tiếng nói hiện đại được sử dụng trong đồ án. Khác với mạng hồi quy (RNN/LSTM) phải xử lý chuỗi một cách tuần tự, Transformer xử lý song song toàn bộ chuỗi đầu vào và mô hình hóa quan hệ giữa các phần tử bằng cơ chế chú ý (attention), nhờ đó huấn luyện nhanh hơn và nắm bắt phụ thuộc tầm xa tốt hơn^[10].

Cơ chế Self-Attention. Mỗi token đầu vào được chiếu thành ba vector Query (Q), Key (K) và Value (V). Mức độ chú ý của một token tới các token khác được tính bằng tích vô hướng có tỷ lệ giữa Q và K, chuẩn hóa bằng hàm softmax, rồi dùng để lấy trung bình có trọng số các vector V theo công thức:

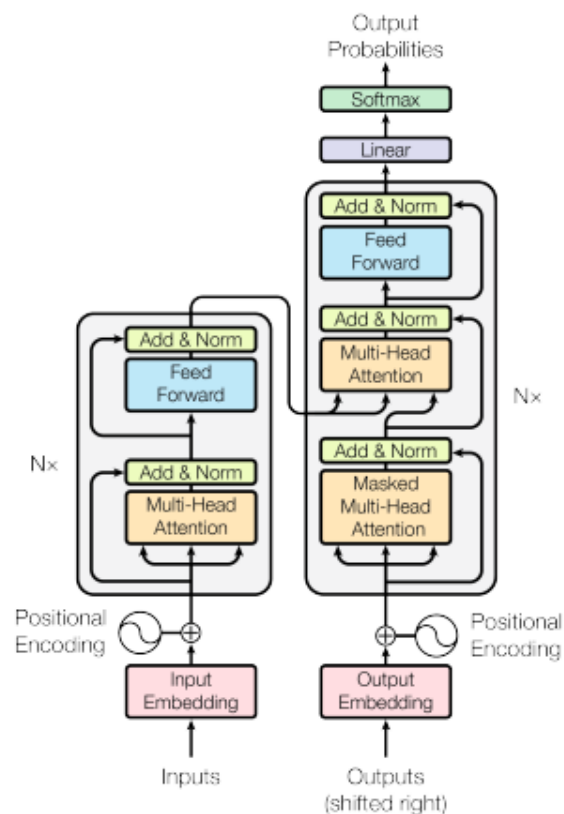
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) \cdot V$$

trong đó d_k là số chiều của vector Key; hệ số $\sqrt{d_k}$ giúp ổn định gradient. Cơ chế này cho phép mỗi token tổng hợp thông tin từ mọi token khác trong chuỗi tùy theo mức độ liên quan ngữ nghĩa^[14].

Multi-Head Attention. Thay vì tính một attention duy nhất, Transformer chạy song song nhiều "đầu" attention (head), mỗi đầu học một kiểu quan hệ khác nhau (cú pháp, ngữ nghĩa, vị trí...), sau đó ghép kết quả lại, giúp mô hình nắm bắt đồng thời nhiều khía cạnh của ngữ cảnh^[14].

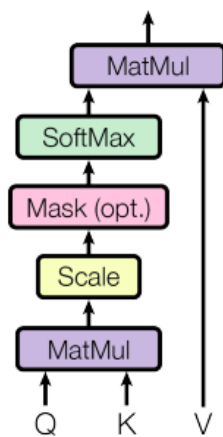
Positional Encoding. Vì attention không có khái niệm thứ tự, Transformer cộng thêm thông tin vị trí (positional encoding) vào embedding đầu vào để mô hình biết thứ tự các token^[14].

Khối Transformer. Một khối gồm lớp Multi-Head Attention, mạng truyền thẳng theo từng vị trí, kèm kết nối tắt (residual) và chuẩn hóa lớp (layer normalization). Xếp chồng nhiều khối tạo nên mô hình sâu. Trong VLM, các token thị giác (sau khi qua vision encoder và projector) được nối với token văn bản rồi cùng đi qua chồng khối Transformer của LLM — đây chính là cơ chế cho phép mô hình đọc hiểu hình ảnh trong ngữ cảnh câu hỏi^[14].

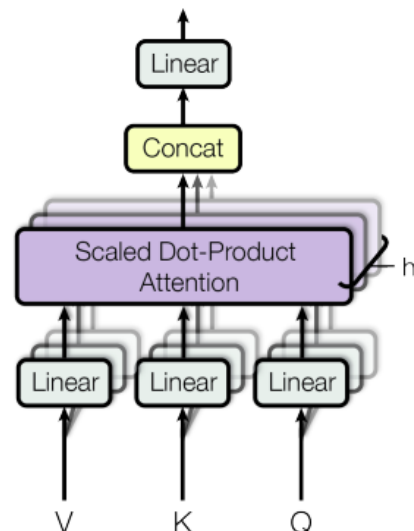


Hình 2.1: Sơ đồ kiến trúc khối Transformer_[12]

Scaled Dot-Product Attention



Multi-Head Attention



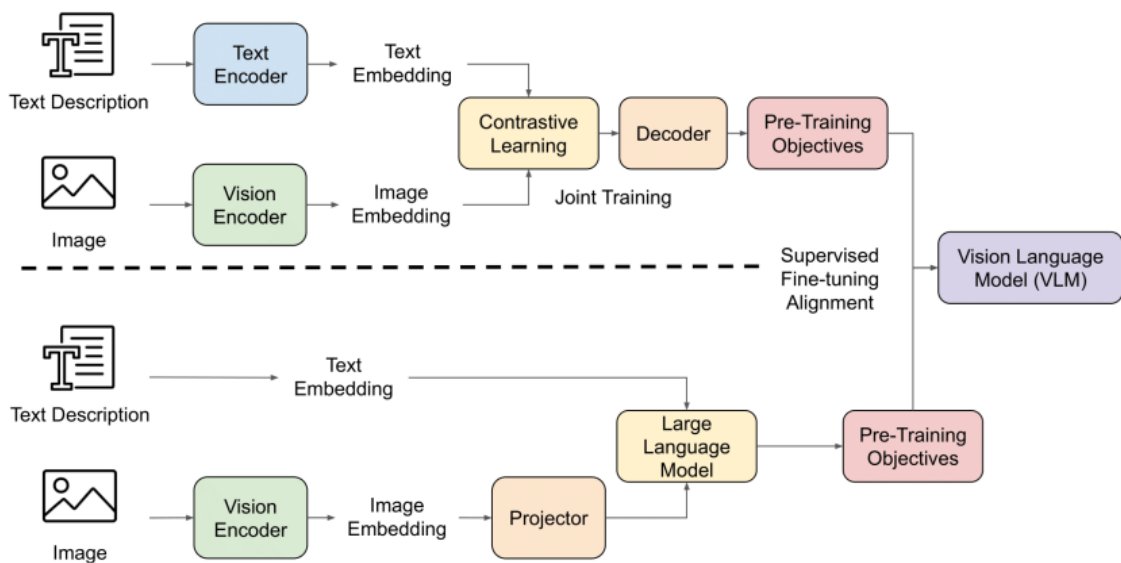
Hình 2.2: Cơ chế Self-attention và Multi-Head_[12]

2.2. Kiến trúc Vision-Language Model (VLM)

Mô hình Ngôn ngữ - Thị giác (Vision-Language Model - VLM) là lớp mô hình học sâu đa phương thức có khả năng tiếp nhận đồng thời hai loại dữ liệu đầu vào là hình ảnh

và văn bản, từ đó sinh ra phản hồi bằng ngôn ngữ tự nhiên. Khác với các mô hình thị giác truyền thống chỉ thực hiện phân loại hoặc phát hiện đối tượng, VLM có khả năng "đọc hiểu" nội dung hình ảnh trong mối liên hệ với chỉ dẫn dạng văn bản, nhờ đó phù hợp với các bài toán đòi hỏi suy luận theo ngữ cảnh như mô tả ảnh, hỏi đáp trực quan (Visual Question Answering) và dịch thuật có ngữ cảnh hình ảnh [10][3].

Về mặt kiến trúc, một mô hình VLM điển hình gồm ba thành phần chính: (1) bộ mã hóa thị giác (Vision Encoder) như CLIP hoặc SigLIP, có nhiệm vụ chia ảnh thành các mảnh nhỏ (patch) và chuyển thành chuỗi token thị giác; (2) bộ chiếu/kết nối (Projector/Connector) thường là một mạng MLP hoặc Q-Former, ánh xạ các token thị giác vào không gian biểu diễn của mô hình ngôn ngữ; và (3) mô hình ngôn ngữ lớn (LLM) đóng vai trò bộ suy luận trung tâm, nhận chuỗi token thị giác đã được chiếu rồi ghép cùng token văn bản để sinh ra kết quả. Hai hướng tích hợp phổ biến là kiến trúc tự hồi quy hoàn toàn (chiếu token thị giác vào đầu vào của LLM) và kiến trúc cross-attention (chèn đặc trưng thị giác vào các tầng bên trong LLM) [10].



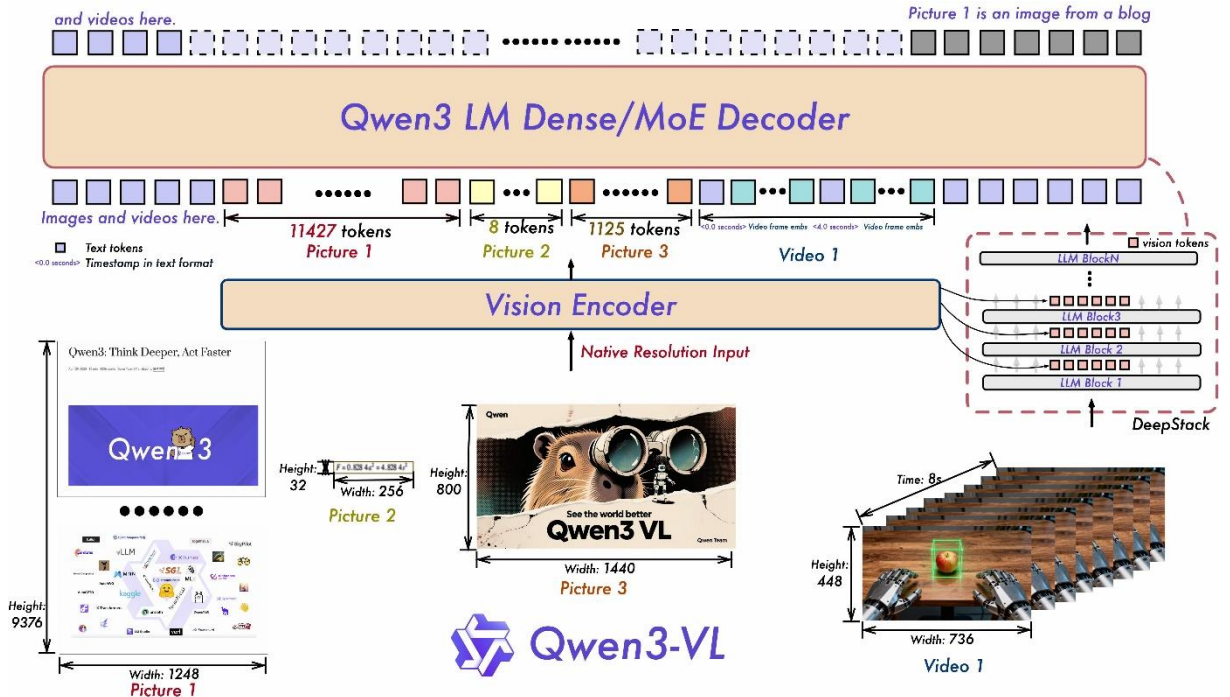
Hình 2.3: Kiến trúc tổng quát của Vision-Language Model (VLM)[10]

Hệ thống sử dụng hai dòng mô hình VLM chính để phục vụ các bài toán phân loại và dịch thuật có ngữ cảnh:

API Gemini 2.5 Flash Lite (Cloud): Hỗ trợ cửa sổ ngữ cảnh cực lớn (1 triệu tokens), có khả năng hiểu video bản gốc bằng cách truyền trực tiếp file video qua API. Gemini được sử dụng khi hệ thống hoạt động ở chế độ API nhờ tốc độ suy diễn nhanh và khả năng dịch thuật tự nhiên xuất sắc [6].

Qwen3.5-2B (Local): Là mô hình ngôn ngữ lớn đa phương thức chạy local mặc định của hệ thống. Qwen3.5-2B có kích thước nhỏ gọn (khoảng 2 tỷ tham số), tiêu thụ bộ nhớ VRAM thấp (~4GB), phù hợp chạy trên các dòng GPU phổ thông của máy cá nhân. Mô hình sử dụng chat template chuẩn của thư viện Transformers và lớp AutoModelForImageTextToText, giúp dễ dàng tích hợp và giải phóng bộ nhớ (unload) linh hoạt để nhường VRAM cho các module khác.

Qwen3.5-2B thuộc dòng mô hình thị giác - ngôn ngữ Qwen-VL do Alibaba phát triển, được tổ chức theo kiến trúc ba mô-đun gồm bộ mã hóa thị giác (Vision Encoder dựa trên SigLIP2), bộ trộn thị giác - ngôn ngữ (Vision-Language Merger dạng MLP) nén đặc trưng ảnh thành token thị giác, và bộ giải mã ngôn ngữ (LLM decoder). Nhờ số lượng tham số nhỏ (khoảng 2 tỷ) và cửa sổ ngữ cảnh dài, mô hình vừa đủ năng lực hiểu hình ảnh khung hình, vừa có thể chạy cục bộ trên GPU phổ thông [2][7].



Hình 2.4: Kiến trúc dòng mô hình Qwen-VL (Qwen3.5-2B, chạy local)[7]

2.3. Nhận dạng ký tự quang học và mô hình GLM-OCR

Nhận dạng ký tự quang học (OCR) là bài toán chuyển vùng ảnh chứa chữ thành chuỗi ký tự, thường gồm hai giai đoạn: phát hiện vùng văn bản (text detection) xác định các hộp bao (bounding box) chứa chữ, và nhận dạng văn bản (text recognition) giải mã chuỗi ký tự trong từng hộp[9].

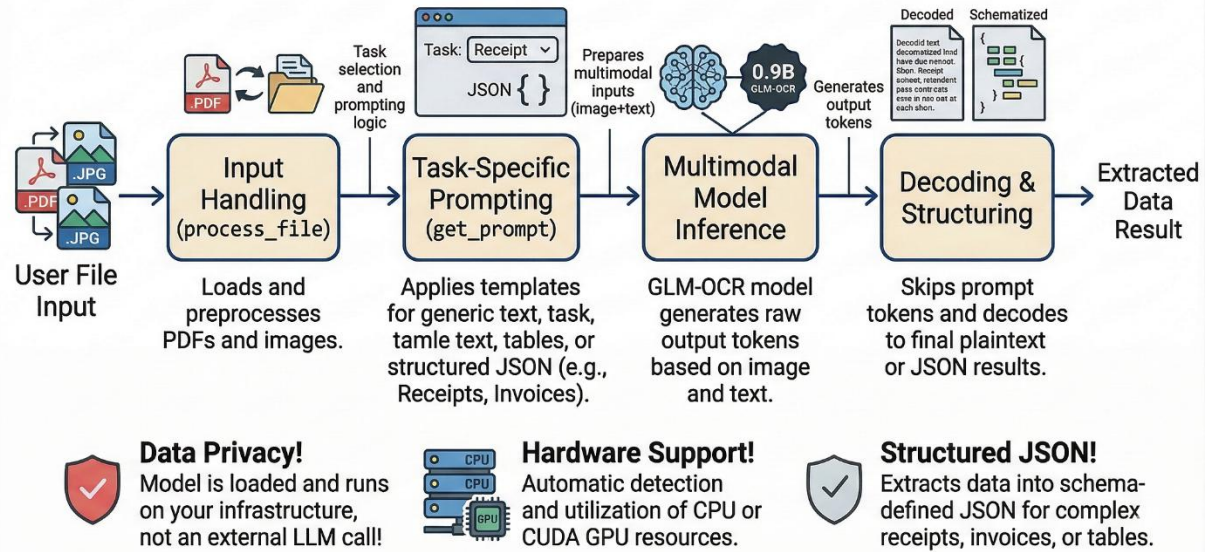
Giải mã dựa trên CTC (Connectionist Temporal Classification): mô hình dự đoán xác suất ký tự tại mỗi cột đặc trưng của ảnh, rồi dùng hàm mất mát CTC để căn chỉnh chuỗi dự đoán với nhãn mà không cần biết trước vị trí từng ký tự; phù hợp văn bản một dòng, tốc độ cao^[9].

Giải mã dựa trên Attention/Seq2Seq: bộ giải mã sinh ký tự tuần tự, mỗi bước chú ý đến vùng ảnh liên quan; linh hoạt hơn với bố cục phức tạp. Các mô hình OCR đa phương thức mới (như GLM-OCR) đi theo hướng xem OCR là bài toán sinh văn bản có điều kiện trên ảnh, nhờ đó đọc tốt cả khi nền phức tạp hay chữ hiển thị dạng động — rất phù hợp với phụ đề gắn sẵn (hard-sub) trong đồ án^[9].

Để thu thập chính xác vị trí và nội dung phụ đề gắn sẵn trên video, hệ thống self-host mô hình GLM-OCR. Đây là mô hình OCR đa phương thức tối ưu cho việc nhận dạng chữ viết trong các môi trường hình ảnh phức tạp hoặc tài liệu số. GLM-OCR có khả năng định vị chính xác vùng văn bản (Bounding Boxes) và nhận dạng ký tự tiếng Anh/tiếng Việt với độ chính xác cao ngay cả khi chữ hiển thị dạng hoạt ảnh.

Về mặt kiến trúc, GLM-OCR là mô hình thị giác - ngôn ngữ nhỏ gọn (khoảng 0,9 tỷ tham số) gồm bộ mã hóa thị giác CogViT, một bộ kết nối đa phương thức nhẹ có cơ chế giảm số lượng token (token downsampling) và bộ giải mã ngôn ngữ GLM-0.5B. Mô hình được huấn luyện với hàm mất mát Multi-Token Prediction (MTP) kết hợp học tăng cường (Reinforcement Learning), đạt kết quả dẫn đầu (SOTA) trên bộ đánh giá phân tích tài liệu OmniDocBench V1.5 và có thể sinh đầu ra ở các định dạng có cấu trúc như Markdown, JSON hoặc LaTeX ^[9].

End-to-End Workflow of the GLMOCR Document Understanding Agent

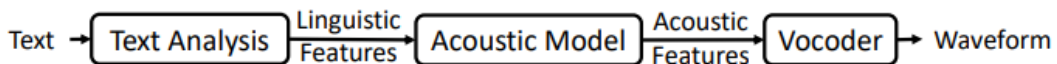


Hình 2.5: Kiến trúc mô hình GLM-OCR^[13]

2.4. Công nghệ nhân bản giọng nói Zero-shot VieNeu-TTS v2 Turbo

Công nghệ tổng hợp tiếng nói (Text-to-Speech) đã trải qua ba thế hệ: tổng hợp ghép nối (concatenative) ghép các đoạn âm thanh thu sẵn; tổng hợp tham số thống kê (parametric) dùng mô hình sinh tham số âm học rồi tổng hợp bằng vocoder; và neural TTS dùng mạng nơ-ron sâu cho chất lượng gần giọng người^[15].

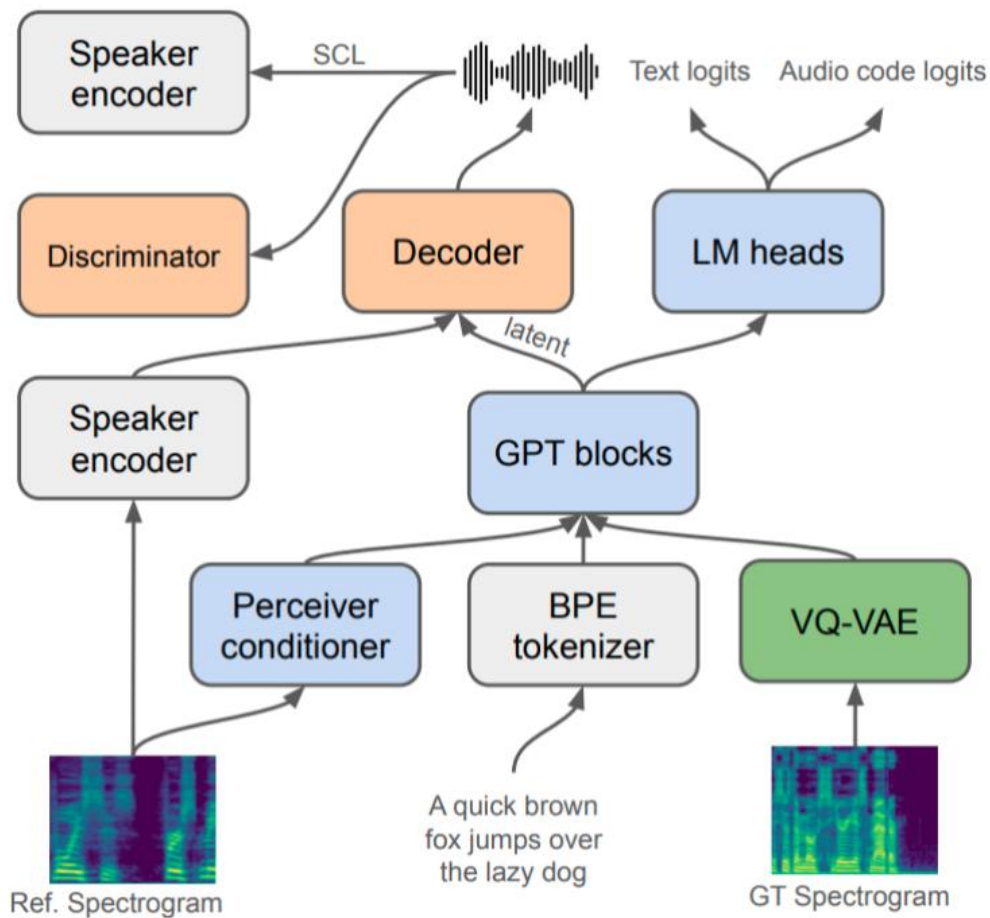
Một hệ neural TTS điển hình gồm hai phần: mô hình âm học (acoustic model) ánh xạ văn bản/âm vị sang biểu diễn âm học (thường là mel-spectrogram), và vocoder chuyển biểu diễn âm học thành sóng âm. Các mốc tiêu biểu gồm Tacotron 2, FastSpeech 2 (phi hồi quy, suy diễn nhanh), VITS (end-to-end) và xu hướng dùng neural audio codec để mô hình ngôn ngữ sinh trực tiếp token âm thanh^{[1][15]}.



Hình 2.6: Pipeline Neural TTS^[15]

Khái niệm Zero-shot Voice Cloning: Là kỹ thuật tổng hợp giọng nói từ một văn bản bất kỳ bằng cách sao chép đặc trưng giọng nói của một người khác chỉ từ một đoạn âm thanh mẫu rất ngắn (3-10 giây) mà không cần huấn luyện lại toàn bộ mô hình (fine-tuning). Hướng này bổ sung một bộ mã hóa người nói (speaker encoder) trích đặc trưng giọng từ một mẫu rất ngắn để điều kiện hóa quá trình sinh, giữ nguyên màu giọng mà

không cần huấn luyện lại — đúng hướng mà mô hình VieNeu-TTS v2 trong đồ án áp dụng [1][8][11].



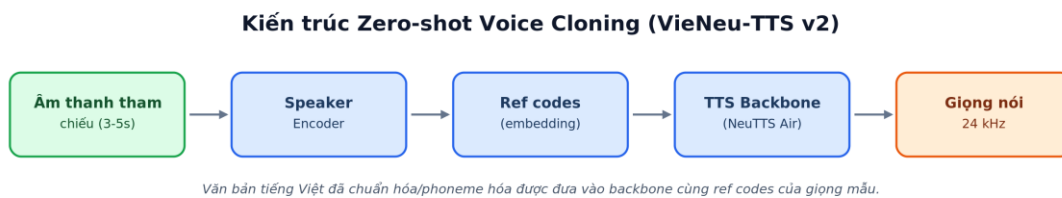
Hình 2.7: Cơ chế zero-shot voice-cloning theo XTTS_[1]

VieNeu-TTS v2 Turbo: Là engine TTS cốt lõi được chọn cho hệ thống. Mô hình được xây dựng dựa trên kiến trúc tổng hợp tiếng nói tốc độ cao, hỗ trợ tiếng Việt bản xứ tự nhiên. Đầu ra của mô hình là âm thanh thuyết minh dạng mảng numpy float32 mono với tần số lấy mẫu mặc định là 24kHz_[8].

VieNeu-TTS v2 được phát triển bằng cách tinh chỉnh (fine-tune) trên kiến trúc NeuTTS Air và huấn luyện với kho ngữ liệu tiếng Việt quy mô lớn (phiên bản v2 sử dụng hơn 10.000 giờ dữ liệu song ngữ), cho phép nhân bản giọng nói tức thời chỉ với 3-5 giây âm thanh tham chiếu. Mô hình hỗ trợ tổng hợp giọng nói ngay trên thiết bị (on-device) theo thời gian thực trên cả CPU lẫn GPU, với chất lượng âm thanh 24 kHz và một chế độ hội thoại/podcast chuyên biệt, rất phù hợp để tổng hợp các đoạn thuyết minh dài _[8].

Cơ chế hoạt động:

- Đầu tiên, file âm thanh tham chiếu (Reference Audio) được đưa qua bộ mã hóa để trích xuất vector đặc trưng giọng nói (ref codes).
- Văn bản tiếng Việt đã dịch được chuẩn hóa kỹ lưỡng (loại bỏ dấu câu thừa, chuyển đổi số thành chữ đọc tự nhiên).
- Mô hình nhận ref codes và văn bản để tổng hợp ra tệp âm thanh thuyết minh có màu giọng tương đồng với giọng mẫu.
- Hệ thống sử dụng cơ chế lưu trữ đệm ref codes trong bộ nhớ (In-memory cache) thông qua mã băm SHA1 của file âm thanh tham chiếu, giúp tránh việc phải encode lại nhiều lần, tăng tốc độ xử lý hàng loạt.



Hình 2.8: Kiến trúc Zero-shot Voice Cloning (VieNeu-TTS v2) được vẽ lại

2.5. Thư viện đồng bộ và xử lý âm thanh Rubberband & Pydub

Vấn đề cốt lõi của bài toán đồng bộ là phải thay đổi thời lượng đoạn audio tiếng Việt cho khớp khung thời gian phụ đề mà không làm thay đổi cao độ (pitch) — nếu chỉ phát nhanh/chậm như tua băng thì giọng sẽ bị méo.

Phase Vocoder. Kỹ thuật này làm việc trên miền tần số: tín hiệu được phân tích bằng biến đổi Fourier thời gian ngắn (STFT) thành các khung phổ; thuật toán nội suy/giãn các khung theo thời gian và hiệu chỉnh pha để giữ tính liên tục, rồi tổng hợp ngược về miền thời gian. Nhờ đó tốc độ phát thay đổi nhưng tần số cơ bản (cao độ) được bảo toàn. Công cụ Rubberband mà đề án sử dụng hiện thực kỹ thuật này ở chất lượng cao, là cơ sở cho thuật toán Co giãn có Ngân sách ở mục 3.4^[17].

Rubberband CLI: Là một thư viện xử lý tín hiệu âm thanh chất lượng cao chuyên dụng cho tác vụ co giãn thời lượng âm thanh (Time-Stretching) và dịch chuyển cao độ (Pitch-Shifting). Rubberband hoạt động trên miền tần số (Phase Vocoder), giúp thay đổi tốc độ phát của giọng nói thuyết minh tiếng Việt mà không làm biến đổi cao độ giọng, giữ nguyên sự tự nhiên và sắc thái giọng đọc của mô hình TTS^[17].

Pydub: Thư viện Python dùng để xử lý các tệp âm thanh ở mức cơ bản như cắt ghép, điều chỉnh âm lượng, và chèn thêm các đoạn lặng (silence padding) vào đầu/cuối tệp tin để đồng bộ chính xác với timeline hiển thị của phụ đề^[18].

2.6. Biểu diễn và đo lường đặc trưng giọng nói (Speaker Embedding)

Để đánh giá mức độ giống nhau giữa giọng thuyết minh sinh ra và giọng mẫu, cần biểu diễn mỗi đoạn tiếng nói bằng một vector đặc trưng người nói (speaker embedding). Các kỹ thuật phổ biến gồm d-vector và x-vector, được huấn luyện cho bài toán xác minh người nói sao cho hai đoạn của cùng một người nằm gần nhau trong không gian vector, khác người thì cách xa nhau^[16].

Đo độ tương đồng. Đồ án dùng thư viện Resemblyzer để trích embedding và đo độ tương đồng bằng cosine similarity giữa hai vector u và v :

$$\cos(u, v) = (u \cdot v) / (\|u\| \cdot \|v\|)$$

Giá trị nằm trong khoảng $[-1, 1]$; càng gần 1 thì hai giọng càng giống nhau^[16]. Ngưỡng 0.75 được chọn làm mốc đạt độ tương đồng giọng nói trong đồ án.

2.7. Các độ đo đánh giá chất lượng

Đồ án sử dụng kết hợp nhiều độ đo định lượng để đánh giá toàn diện chất lượng đầu ra của hệ thống tích hợp:

BLEU-4: đo độ trùng khớp n-gram (đến bậc 4) giữa bản dịch máy và bản tham chiếu, kèm hệ phạt độ dài; giá trị 30–45 được xem là dịch tốt với văn bản kỹ thuật ^[4].

chrF++: dựa trên F-score của n-gram ký tự và từ, ít nhạy với khác biệt phân tách từ, phù hợp tiếng Việt.

Speaker Similarity: cosine similarity của speaker embedding (mục 2.6).

MOS (Mean Opinion Score): điểm đánh giá chủ quan trung bình về độ tự nhiên của giọng (thang 1–5), có thể ước lượng tự động bằng mô hình học sâu.

Độ chính xác đồng bộ: Mean/p95 Delay (sai lệch thời điểm bắt đầu phát âm so với mốc phụ đề) và các chỉ số Caption Drift (Start/End Drift, Duration Jaccard).

Lưu ý: do đồ án thiên về ứng dụng và không xây dựng bộ bản dịch tham chiếu chuẩn, phần đánh giá chất lượng dịch thuật ở Chương 4 sử dụng phương pháp chấm thủ công theo rubric (Adequacy và Fluency, thang 1–5) thay cho BLEU/chrF++. Hai chỉ số BLEU-4 và chrF++ nêu trên chỉ được trình bày như kiến thức nền về đo lường chất lượng dịch máy tự động.

2.8. Lựa chọn công nghệ cho hệ thống và so sánh phương án

Hệ thống của đề án không phát triển mô hình mới mà tích hợp các mô hình và công cụ có sẵn; do đó việc lựa chọn đúng công nghệ cho từng khâu mang tính quyết định đối với chất lượng đầu ra. Mục này làm rõ các công nghệ thực sự được sử dụng, các phương án thay thế đã cân nhắc và lý do lựa chọn.

Thành phần	Công nghệ được chọn	Phương án thay thế	Lý do lựa chọn
Trục thời gian phụ đề	OCR-mode (GLM-OCR)	VLM-mode (VLM tự sinh mốc thời gian)	VLM dễ ảo giác mốc thời gian; OCR bám sát phụ đề thật nên ổn định hơn
Mô hình thị giác - ngôn ngữ	Qwen3.5-2B (local) + Gemini 2.5 Flash Lite (API)	LLaVA, GPT-4V (API trả phí)	Qwen chạy cục bộ, miễn phí, đủ năng lực; Gemini bổ sung khi cần ngữ cảnh dài
Nhận dạng phụ đề (OCR)	GLM-OCR	Tesseract, PaddleOCR, EasyOCR	Đọc tốt chữ động/nền phức tạp, xuất kết quả có cấu trúc, độ chính xác cao
Tổng hợp giọng (TTS)	VieNeu-TTS v2 (zero-shot)	XTTS/Coqui, F5-TTS	Hỗ trợ tiếng Việt bản xứ, nhân bản giọng từ mẫu 3–5 giây, chạy on-device
Đồng bộ thời lượng	Rubberband (Phase Vocoder)	Tăng/giảm tốc đơn thuần, WSOLA	Giữ nguyên cao độ khi co giãn thời lượng, chất lượng giọng tốt

Bảng 2.1: Bảng so sánh các lựa chọn công nghệ chính của hệ thống

Ở khâu trích xuất, đề án ưu tiên độ chính xác mốc thời gian nên chọn OCR làm trục thời gian cơ sở thay vì để VLM tự sinh timestamp — vốn dễ gặp hiện tượng ảo giác; VLM chỉ được giữ lại cho hai việc nó làm tốt là lọc nhiễu lời thoại và dịch thuật có ngữ cảnh.

Ở khâu tổng hợp giọng, yếu tố quyết định là chất lượng tiếng Việt bản xứ và khả năng nhân bản giọng tức thời từ một mẫu ngắn, đồng thời chạy được trên GPU phổ thông; VieNeu-TTS v2 đáp ứng tốt các tiêu chí này so với các engine đa ngôn ngữ như XTTS. Ở khâu đồng bộ, Rubberband được chọn vì co giãn được thời lượng mà vẫn giữ nguyên cao độ giọng — điều mà cách tăng/giảm tốc độ phát đơn thuần không làm được.

Việc ưu tiên các mô hình tự host (self-hosted) và công cụ mã nguồn mở còn giúp giảm chi phí vận hành, chủ động về dữ liệu và dễ tái lập kết quả, phù hợp với phạm vi của một đề án tốt nghiệp.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể và luồng dữ liệu (Data Pipeline)

Hệ thống được thiết kế theo kiến trúc mô-đun hóa nhằm tách biệt rõ ràng các giai đoạn xử lý, giúp dễ bảo trì, dễ thay thế mô hình và quản lý tài nguyên GPU hiệu quả. Toàn bộ hệ thống gồm ba mô-đun xử lý chính, mỗi mô-đun đảm nhận một nhóm chức năng riêng biệt:

Mô-đun Trích xuất và Dịch thuật (Module 1 - VLM/OCR): Chịu trách nhiệm lấy mẫu khung hình, dùng GLM-OCR để trích xuất phụ đề tiếng Anh sẵn, dùng VLM để lọc bỏ văn bản nhiễu trên màn hình và dịch thuật theo ngữ cảnh sang tiếng Việt (chi tiết tại mục 3.2).

Mô-đun Tổng hợp Giọng nói (Module 2 - TTS): Nhận văn bản tiếng Việt đã dịch cùng tệp âm thanh tham chiếu, sử dụng mô hình VieNeu-TTS để nhân bản và tổng hợp giọng thuyết minh tiếng Việt cho từng phân đoạn (chi tiết tại mục 3.3).

Mô-đun Đồng bộ và Render (Module 3 - Sync/Render): Căn chỉnh thời lượng các đoạn audio cho khớp với timeline phụ đề bằng thuật toán co giãn có ngân sách và cơ chế trượt tích lũy, sau đó trộn âm và ghép trở lại video gốc bằng FFmpeg (chi tiết tại mục 3.4).

Ba mô-đun trên được điều phối tuần tự bởi module PipelineRunner, tạo thành một đường ống xử lý dữ liệu (data pipeline) gồm 8 bước. Sơ đồ luồng dữ liệu tổng thể của hệ thống được mô tả như Hình 3.1:

Sơ đồ luồng dữ liệu pipeline xử lý



Hình 3.1: Sơ đồ luồng dữ liệu pipeline 8 bước ở chế độ OCR

3.2. Thiết kế Module Trích xuất và Dịch thuật (Module 1 - VLM/OCR)

3.2.1. Quyết định kỹ thuật: Từ bỏ VLM-mode chuyển sang OCR-mode

Trong các giai đoạn thiết kế ban đầu, hệ thống hỗ trợ cả chế độ trích xuất tự động hoàn toàn bằng VLM (VLM Mode). Trong chế độ này, mô hình VLM được yêu cầu nhìn các khung hình được chọn thích ứng sau khi phân cảnh, tự mô tả hành động trên màn hình, tự biên soạn lời thoại tiếng Anh, tự dịch sang tiếng Việt và tự gán mốc thời

gian bắt đầu/kết thúc (Timestamp). Việc phân cảnh thích ứng dựa trên công cụ phát hiện chuyển cảnh theo nội dung [5].

Tuy nhiên, kết quả thực nghiệm cho thấy phương pháp này gặp phải những hạn chế nghiêm trọng:

- Mô hình VLM thường xuyên gặp hiện tượng ảo giác thời gian, gán các mốc thời gian không có thật hoặc dồn tất cả các phụ đề được tạo ra vào một phần nhỏ của video.
- Khi lặp lại các chunk, VLM thường bị lặp nội dung thuyết minh hoặc bỏ sót các sự kiện diễn ra nhanh trên màn hình.

Vì vậy, hệ thống đã chuyển đổi kiến trúc lấy **OCR-mode** làm cốt lõi để xử lý các video bắt buộc phải có phụ đề tiếng Anh gắn sẵn (hard-sub). Quy trình này sử dụng OCR chuyên dụng để thiết lập một trục thời gian (Timeline) cơ sở cực kỳ chính xác trước, sau đó mới dùng VLM để dịch thuật ngữ nghĩa.

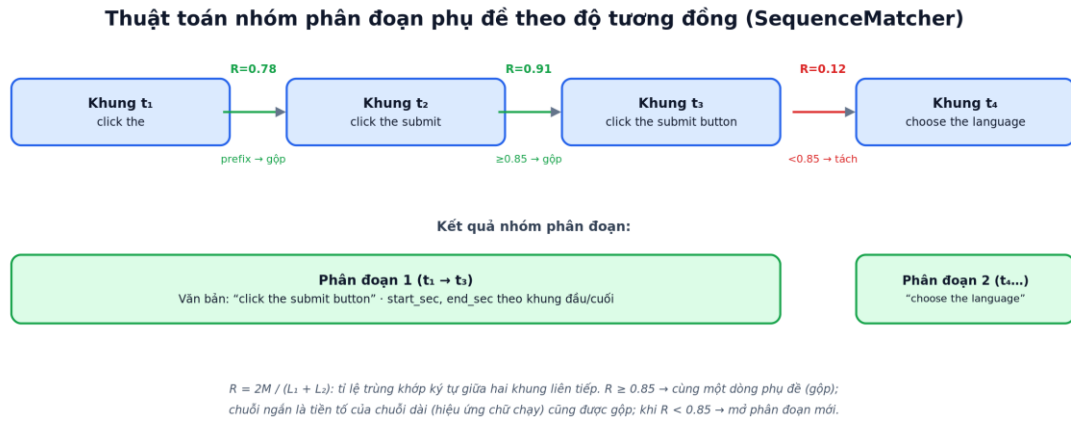
3.2.2. Trích xuất Timeline Phụ đề bằng CaptionTimeline

Module CaptionTimeline thực hiện lấy mẫu video một cách tuần tự (Sequential Decoding) với tốc độ lấy mẫu mặc định là 3 FPS. Việc giải mã tuần tự giúp bắt kịp các chuyển đổi phụ đề nhanh mà không bị bỏ sót khung hình.

Quy trình trích xuất và nhóm thời gian chi tiết của thuật toán như sau:

- **Trích xuất vùng chứa:** Tại mỗi khung hình được trích xuất (tương ứng với thời điểm t_i), hệ thống cắt vùng chứa phụ đề ở phía dưới màn hình theo tỷ lệ cấu hình (mặc định là 10% chiều cao dưới cùng của video).
- **Nhận dạng:** Chạy mô hình GLM-OCR[9] trên vùng cắt để trích xuất chuỗi ký tự tiếng Anh T_i .
- **Thuật toán nhóm phân đoạn:** Hệ thống duy trì một phân đoạn hiện tại. Đối với mỗi khung hình mới có văn bản T_{i+1} , hệ thống so sánh với văn bản T_i của khung hình trước đó như sau:
 - **Tính độ tương đồng chuỗi:** Sử dụng thuật toán SequenceMatcher (Gestalt Pattern Matching) để tính tỉ lệ tương đồng R giữa hai chuỗi theo công thức $R = \frac{2M}{(L_1 + L_2)}$, trong đó M là số ký tự trùng khớp chung, L_1 và L_2 là chiều dài hai chuỗi[22]. Nếu $R \geq 0.85$, hệ thống coi hai khung hình chứa cùng một dòng phụ đề và tiếp tục gộp vào phân đoạn hiện tại.

- **Xử lý hiệu ứng chữ chạy (Typewriter/Animation):** Nếu một chuỗi ngắn là tiền tố của chuỗi dài hơn xuất hiện liền kề (ví dụ: “click the” và “click the submit button”), hệ thống sẽ gộp và cập nhật nội dung văn bản của phân đoạn thành chuỗi dài hơn.
- **Xác định mốc thời gian:** Khi phát hiện một dòng văn bản mới hoàn toàn ($R < 0.85$), phân đoạn cũ sẽ được đóng lại. Thời gian bắt đầu (start_sec) được gán bằng thời điểm khung hình đầu tiên xuất hiện dòng chữ, và thời gian kết thúc (end_sec) được gán bằng thời điểm khung hình cuối cùng còn chứa dòng chữ đó.



Hình 3.2: Minh họa thuật toán nhóm phân đoạn phụ đề bằng SequenceMatcher

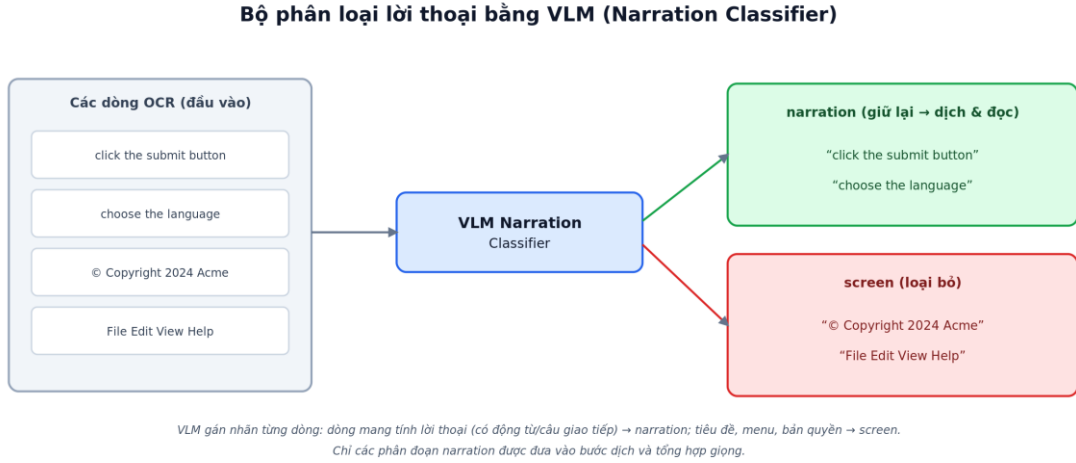
3.2.3. Lọc lời thoại bằng VLM Narration Classifier

OCR sẽ đọc bất kỳ chữ viết nào xuất hiện ở dải dưới cùng của màn hình, bao gồm cả những thông tin bản quyền (ví dụ: “© Copyright” hoặc các nhãn thương hiệu). Giọng thuyết minh không nên phát âm các dòng chữ này.

Hệ thống thiết kế một bộ phân loại nhị phân dựa trên VLM (Narration Classifier). Bộ phân loại nhận danh sách các dòng văn bản đã trích xuất từ OCR và đưa ra nhãn cho mỗi dòng:

- **narration:** Các dòng văn bản có chứa động từ hoặc cấu trúc câu giao tiếp mà một thuyết minh viên sẽ thực tế nói ra (ví dụ: “click the submit button”, “choose the language”).
- **screen:** Các dòng văn bản tĩnh trên màn hình, tiêu đề trang hoặc thông tin bản quyền cần được loại bỏ.

Chỉ những phân đoạn được dán nhãn narration mới được giữ lại đưa vào bước dịch thuật tiếp theo.



Hình 3.3: Ví dụ bộ phân loại VLM Narration Classifier (narration / screen)

3.2.4. Ghép nối Phụ đề ngắn và Kéo dài thời gian kết thúc

A. Thuật toán gộp phân đoạn ngắn (merge_short_segments)

Khi người dùng nói các câu cực ngắn hoặc ngắt quãng, OCR sẽ tạo ra nhiều phân đoạn có thời lượng rất nhỏ. Nếu đưa trực tiếp vào TTS, giọng nói thuyết minh tiếng Việt sinh ra sẽ bị ngắt quãng liên tục, tạo cảm giác thiếu tự nhiên. Thuật toán gộp phân đoạn hoạt động như sau:

- Duyệt qua danh sách các phân đoạn đã được sắp xếp theo thời gian.
- Nếu khoảng cách thời gian giữa điểm kết thúc của phân đoạn S_j và điểm bắt đầu của phân đoạn tiếp theo S_{j+1} nhỏ hơn ngưỡng `max_gap_sec` (mặc định 2.5 giây) VÀ tổng độ dài chuỗi ký tự sau khi gộp nhỏ hơn `max_combined_chars` (mặc định 150 ký tự), hệ thống sẽ gộp chúng thành một phân đoạn duy nhất.
- Phân đoạn mới có thời gian bắt đầu $t_{start} = t_{start}^{(j)}$ và thời gian kết thúc $t_{end} = t_{end}^{(j+1)}$. Nội dung văn bản tiếng Anh được nối lại với nhau bằng dấu cách.

B. Thuật toán kéo dài thời gian kết thúc (extend_end_times)

Để giải quyết vấn đề câu thoại tiếng Việt thường dài hơn tiếng Anh khoảng 30%, hệ thống tự động tăng timing budget của mỗi phân đoạn để tạo thêm khoảng không gian phát âm cho TTS:

- Đối với mỗi phân đoạn S_j , thời gian kết thúc mới: $t_{end,new}^{(j)} = t_{end}^{(j)} + \text{extend_sec}$ (với `extend_sec` mặc định là 0.5 giây).

- Để tránh kéo dài gây chùng lán với phân đoạn tiếp theo S_{j+1} , giá trị kéo dài bị giới hạn bởi: $t_{end,new}^{(j)} \leq t_{start}^{(j+1)} - min_gap_sec$ (min_gap_sec là 0.1 giây).
- Thời điểm kết thúc của phân đoạn cuối cùng cũng được giới hạn để không vượt quá tổng thời lượng video gốc: $t_{end,new}^{(n)} \leq t_{video_dur}$.

Các công thức gộp phân đoạn ngắn (merge_short_segments) và kéo dài thời gian kết thúc (extend_end_times) nêu trên là đề xuất của đồ án dựa trên đặc thù dữ liệu phụ đề video, không kế thừa từ một công trình có sẵn.

3.2.5. Dịch thuật ngữ cảnh VLM từng phân đoạn

Sau khi có danh sách các phân đoạn thoại tiếng Anh sạch kèm mốc thời gian, hệ thống gửi từng dòng thoại cùng hình ảnh khung hình đại diện (mid-frame)_[7] của phân đoạn đó cho VLM. Hình ảnh khung hình giúp VLM có thông tin thị giác thực tế để giải nghĩa chính xác các thuật ngữ mơ hồ trong tiếng Anh (ví dụ: phân biệt “table” trên màn hình là bảng dữ liệu hay cái bàn ăn).

Hệ thống duy trì một cửa sổ trượt chứa các dòng dịch đã hoàn thành trước đó gửi kèm vào prompt để đảm bảo giọng văn dịch luôn nhất quán và mượt mà trong suốt toàn bộ video.

3.3. Thiết kế Module Tổng hợp Giọng nói (Module 2 - TTS)

Module nhân bản giọng nói thuyết minh được thiết kế tối ưu hóa hiệu năng và độ ổn định:

Lazy Loading: Để tiết kiệm bộ nhớ VRAM của GPU, mô hình TTS nặng chỉ được tải vào bộ nhớ khi bắt đầu bước tổng hợp giọng nói, và được giải phóng hoàn toàn khỏi GPU thông qua hàm unload_model() ngay sau khi hoàn thành tạo file âm thanh.

Xử lý tuần tự có chủ đích: Do bộ công cụ phát triển (SDK) của mô hình VieNeu-TTS_[8] không an toàn khi chạy đa luồng (non-thread-safe), hệ thống sử dụng cơ chế xử lý tuần tự (Sequential Batch Inference). Bộ xử lý nạp tệp âm thanh tham chiếu, tạo ra vector ref codes một lần duy nhất, sau đó lần lượt duyệt qua các câu thuyết minh tiếng Việt trong file SRT để sinh ra các tệp audio tương ứng, lưu tạm thời vào thư mục trung gian.

3.4. Thiết kế Module Đồng bộ Âm thanh (Module 3 - Sync/Render)

3.4.1. Thuật toán Co giãn có Ngân sách (Budgeted Time-Stretching)

Khi một tệp âm thanh tiếng Việt được tổng hợp bởi TTS, thời lượng của nó (audio_duration) thường dài hơn khoảng thời gian cho phép của phân cảnh phụ đề tương ứng (target_duration). Module AudioAligner sẽ phân tích và đưa ra một trong bốn chiến lược xử lý:

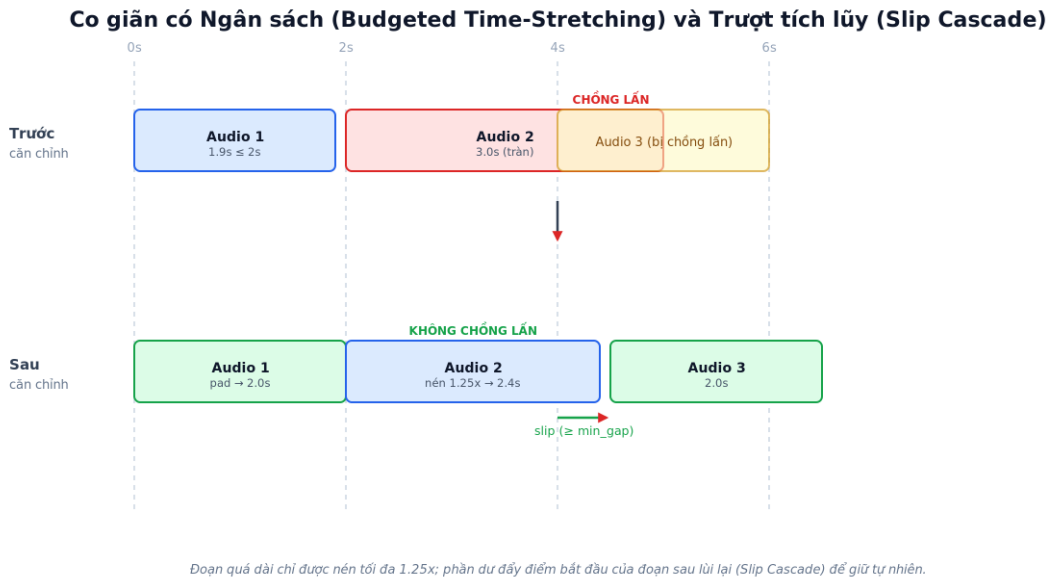
- **exact:** Nếu chênh lệch thời gian nằm trong khoảng dung sai cho phép (mặc định ≤ 0.2 giây), tệp audio được giữ nguyên.
- **pad:** Nếu audio ngắn hơn target, hệ thống sử dụng Pydub^[18] để đệm thêm khoảng lặng vào cuối tệp audio cho vừa vặn.
- **compress_fit:** Nếu audio dài hơn target nhưng tỉ lệ nén cần thiết ≤ 1.25 ($\frac{\text{audio_dur}}{\text{target_dur}} \leq 1.25$), hệ thống gọi Rubberband^[17] để nén tốc độ phát của audio về đúng bằng target_duration mà không thay đổi cao độ.
- **compress_max:** Khi đó, hệ thống chỉ nén đến giới hạn tối đa 1.25x (đạt $\text{achieved_dur} = \frac{\text{audio_dur}}{1.25}$), phần thời gian dư thừa còn lại ($\text{achieved_duration} - \text{target_duration}$) sẽ được xử lý bằng cơ chế trượt tích lũy (Slip Cascade).

3.4.2. Cơ chế Trượt tích lũy (Slip Cascade)

Khi một phân đoạn âm thanh bị tràn thời gian (do áp dụng chiến lược compress_max), thời điểm kết thúc thực tế của nó sẽ lấn sang thời điểm bắt đầu của phân đoạn tiếp theo. Thay vì tiếp tục ép tăng tốc độ phát của phân đoạn sau, thuật toán AudioAligner sẽ dịch chuyển (slip) thời điểm bắt đầu thực tế (effective_start_sec) của phân đoạn sau lùi lại phía sau, đảm bảo có một khoảng trống nhỏ tối thiểu (mặc định ≥ 0.1 giây) giữa hai đoạn tiếng nói.

Cơ chế này cho phép sự quá tải thời lượng của các câu thoại dài lan truyền tự nhiên dọc theo dòng thời gian của video, giúp bảo toàn tối đa chất lượng tự nhiên của giọng đọc thuyết minh.

Thuật toán Co giãn có Ngân sách (Budgeted Time-Stretching) và cơ chế Trượt tích lũy (Slip Cascade) cũng là đề xuất của đồ án; trong đó thao tác co giãn thời lượng mà vẫn giữ nguyên cao độ được thực hiện bằng kỹ thuật Phase Vocoder của công cụ Rubberband^[17], còn công thức đo độ tương đồng chuỗi R sử dụng thuật toán Ratcliff–Obershelp trong thư viện difflib^[22].



Hình 3.4: Minh họa thuật toán Co giãn có Ngân sách và cơ chế Slip Cascade

3.4.3. Trộn âm thanh và Render video bằng FFmpeg

Khắc phục lỗi volume amix: Khi trộn nhiều phân đoạn âm thanh rời rạc lại thành một kênh âm thanh duy nhất, filter amix của FFmpeg_[19] theo mặc định sẽ chia nhỏ âm lượng của từng luồng đầu vào cho tổng số luồng (ví dụ: có 20 phân đoạn thoại thì âm lượng mỗi phân đoạn bị giảm đi 20 lần, khiến video đầu ra có âm thanh rất nhỏ). Hệ thống khắc phục triệt để lỗi này bằng cách cấu hình thuộc tính `normalize=0` trong filter complex của FFmpeg, vì các phân đoạn âm thanh được sắp xếp nối tiếp nhau theo thời gian thực tế nên không có nguy cơ bị chồng lấn hay vỡ âm thanh.

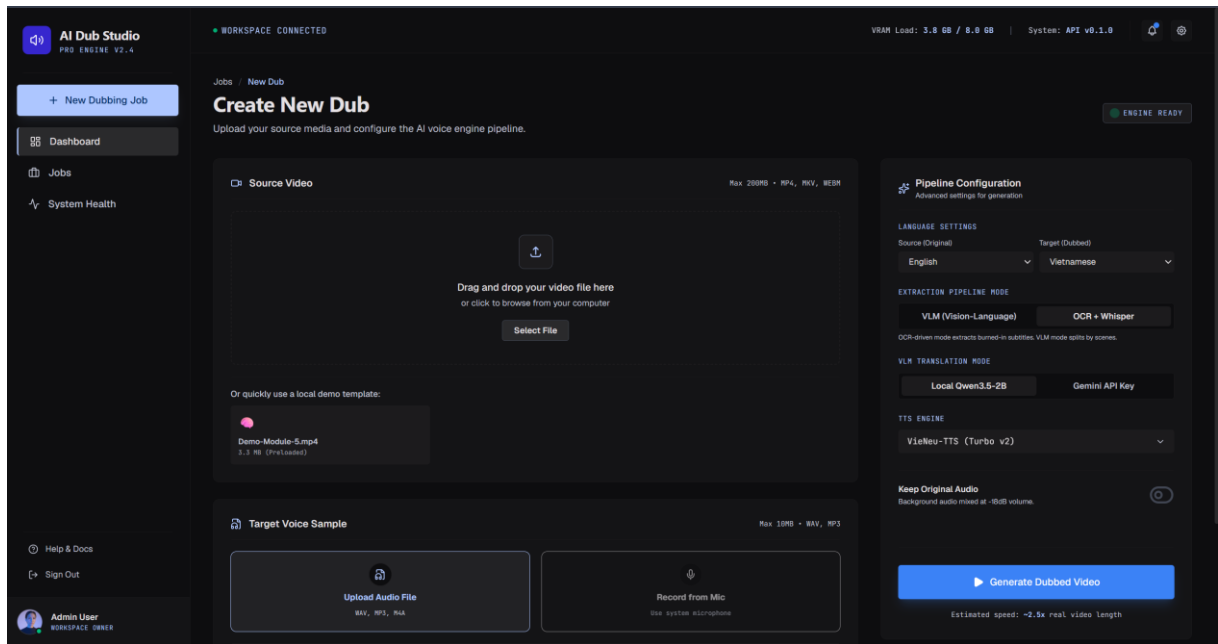
Silence padding và apad: Để tránh việc video bị kết thúc sớm khi chạy lệnh ghép với cờ `-shortest` (do luồng âm thanh kết thúc trước luồng video), hệ thống sử dụng filter `apad` để tự động kéo dài luồng âm thanh bằng khoảng lặng cho đến khi bằng đúng tổng thời lượng của video gốc.

3.5. Thiết kế Hệ thống Web UI (FastAPI + Vite React)

Hệ thống được phát triển với kiến trúc tách biệt rõ ràng giữa Backend và Frontend:

Backend (FastAPI): Cung cấp hệ thống API RESTful_[20] để xử lý việc tải lên tệp tin (`/api/upload`), tạo mới tiến trình xử lý dưới nền (`/api/process`), theo dõi trạng thái tiến độ thời gian thực qua Job ID (`/api/status/{job_id}`), tải xuống kết quả (`/api/result/{job_id}`) và cấu hình hệ thống (`/api/config`). Để đảm bảo tính ổn định và không làm nghẽn luồng xử lý API chính, mỗi job thuyết minh được thực thi trong một tiến trình nền riêng biệt bằng daemon thread.

Frontend (Vite React + Tailwind CSS 4): Cung cấp giao diện Dashboard tương tác hiện đại^[21]. Người dùng có thể dễ dàng tải lên video, tải lên file giọng mẫu, thiết lập các tùy chọn chế độ thuyết minh, theo dõi tiến độ xử lý hiển thị trực quan dưới dạng phần trăm (%) và các dòng nhật ký (logs) chi tiết được đẩy về từ server theo thời gian thực, cuối cùng là xem trước và tải xuống video đã thuyết minh tiếng Việt kèm file phụ đề SRT tương ứng.



Hình 3.5: Giao diện Web Dashboard (FastAPI + Vite React)

3.6. Phân tích yêu cầu hệ thống

Trước khi thiết kế chi tiết, hệ thống được phân tích theo hai nhóm yêu cầu: yêu cầu chức năng (các việc hệ thống phải làm) và yêu cầu phi chức năng (ràng buộc về chất lượng).

Yêu cầu chức năng:

Mã	Chức năng	Mô tả
FR-01	Tải lên video	Tải video tiếng Anh có phụ đề gắn sẵn (hard-sub).
FR-02	Tải lên giọng mẫu	Tải tệp âm thanh tham chiếu 3–10 giây.
FR-03	Cấu hình tiến trình	Chọn chế độ, tỉ lệ vùng cắt, tham số căn chỉnh.
FR-04	Trích xuất & dịch	OCR, lọc nhiễu bằng VLM và dịch sang tiếng Việt.
FR-05	Tổng hợp giọng	Nhân bản giọng mẫu, sinh audio từng phân đoạn.
FR-06	Đồng bộ & ghép video	Căn chỉnh thời lượng, trộn âm, lồng vào video gốc.
FR-07	Theo dõi tiến độ	Hiển thị %, log xử lý theo Job ID.

FR-08	Tải kết quả	Tải video đã thuyết minh và tệp phụ đề SRT.
-------	-------------	---

Bảng 3.1: Danh sách yêu cầu chức năng của hệ thống

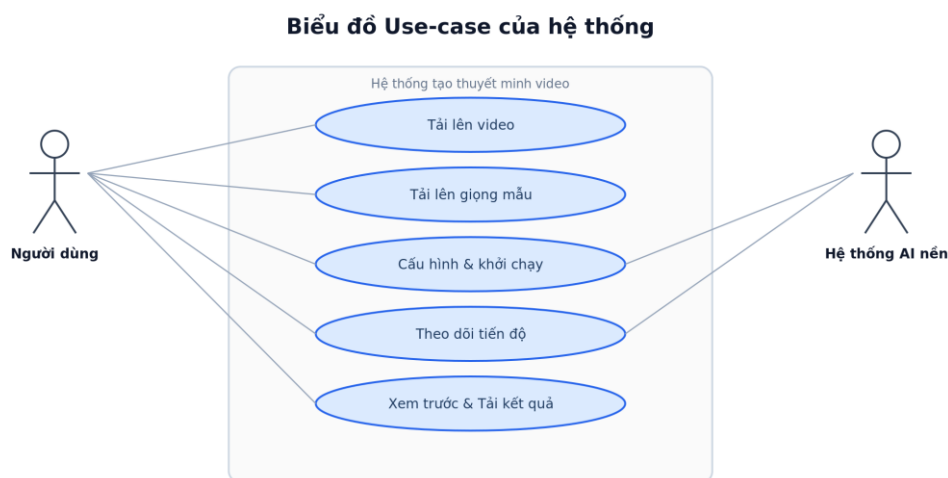
Yêu cầu phi chức năng:

Mã	Tiêu chí	Mục tiêu
NFR-01	Độ tương đồng giọng	Speaker Similarity ≥ 0.75
NFR-02	Độ chính xác đồng bộ	p95 Delay ≤ 0.5 giây
NFR-03	Hiệu năng phần cứng	Chạy được trên GPU 8 GB VRAM
NFR-04	Tính ổn định	Mỗi job chạy trong tiến trình nền độc lập
NFR-05	Khả năng mở rộng	Kiến trúc mô-đun, dễ thay thế mô hình thành phần

Bảng 3.2: Danh sách yêu cầu phi chức năng của hệ thống

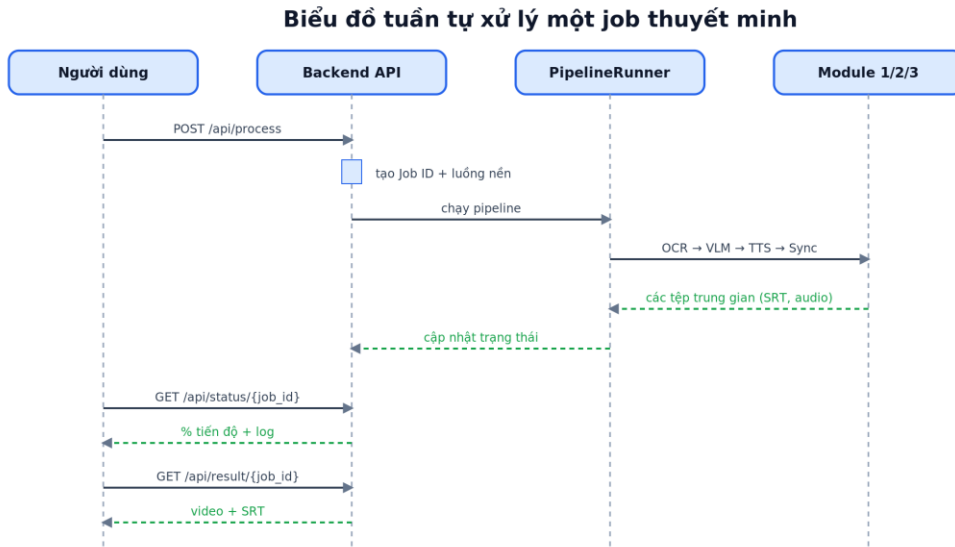
3.7. Mô hình hóa hệ thống

Biểu đồ Use-case. Tác nhân chính là Người dùng (biên tập viên/nhà sản xuất nội dung) tương tác với các use-case: Tải lên video, Tải lên giọng mẫu, Cấu hình & khởi chạy tiến trình, Theo dõi tiến độ, Xem trước và Tải kết quả. Tác nhân phụ là Hệ thống AI nền (các mô hình self-hosted) thực thi OCR, phân loại/dịch VLM, tổng hợp TTS và đồng bộ.



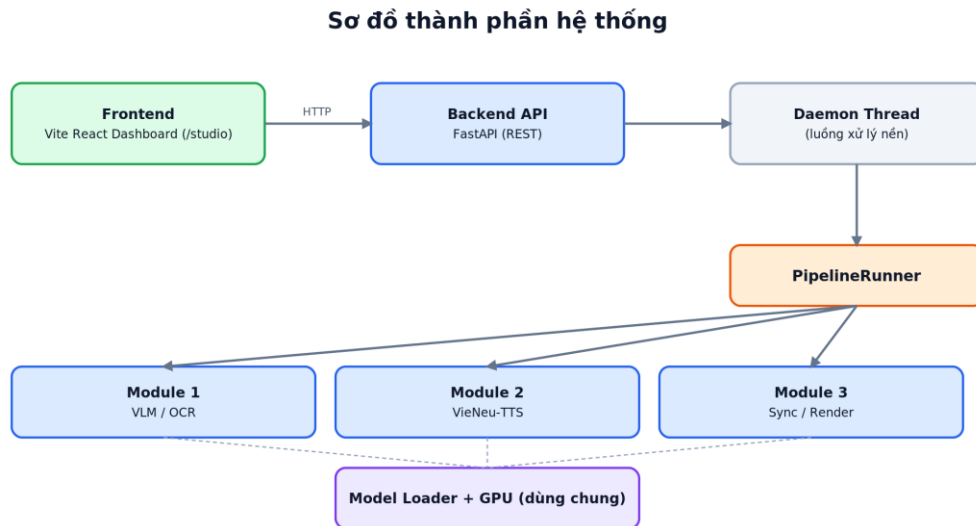
Hình 3.6: Biểu đồ Use-case của hệ thống

Biểu đồ tuần tự (Sequence). Người dùng gửi yêu cầu `/api/process`; Backend tạo Job ID và khởi chạy luồng nền; PipelineRunner lần lượt gọi Module 1 (OCR + VLM), Module 2 (TTS), Module 3 (Sync/Render); trạng thái được cập nhật liên tục để client truy vấn qua `/api/status/{job_id}`; khi hoàn tất, client tải kết quả qua `/api/result/{job_id}`.



Hình 3.7: Biểu đồ tuần tự xử lý một job thuyết minh

Sơ đồ thành phần (Component). Hệ thống gồm Frontend (Vite React Dashboard), Backend API (FastAPI), luồng nền (daemon thread) và ba module xử lý dùng chung tầng nạp mô hình (model loader) với GPU; các module giao tiếp qua tệp trung gian (khung hình, SRT, các đoạn audio) trong thư mục làm việc của job.



Hình 3.8: Sơ đồ thành phần hệ thống

3.8. Thiết kế API và cấu trúc dữ liệu

Backend cung cấp hệ thống API RESTful gồm các endpoint chính:

Phương thức	Endpoint	Chức năng
POST	/api/upload	Tải lên video / giọng mẫu, trả về định danh tệp

POST	/api/process	Khởi tạo job xử lý, trả về job_id
GET	/api/status/{job_id}	Truy vấn tiến độ (%), trạng thái, log
GET	/api/result/{job_id}	Tải video kết quả và tệp SRT
GET/POST	/api/config	Đọc/đặt cấu hình hệ thống

Bảng 3.3: Danh sách các API chính của hệ thống (RESTful)

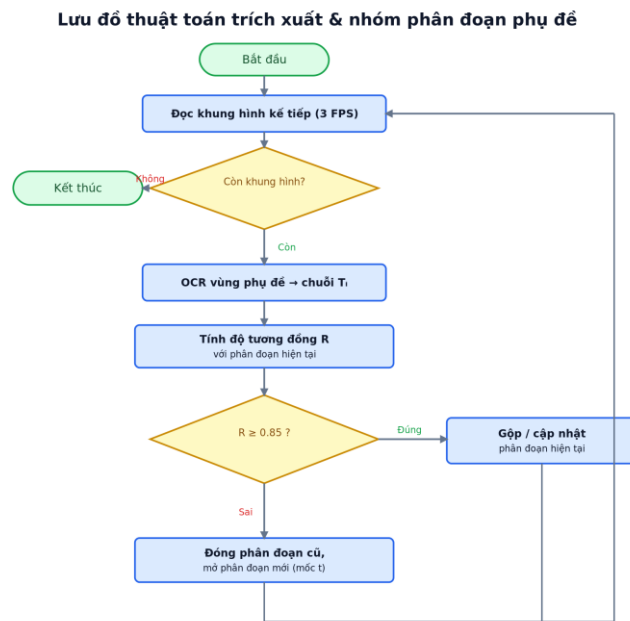
Cấu trúc dữ liệu một phân đoạn (segment): gồm các trường start_sec, end_sec (mốc thời gian), text_en (văn bản OCR), text_vi (bản dịch), label (narration/screen), audio_path (tệp audio đã tổng hợp) và effective_start_sec (mốc bắt đầu thực tế sau Slip Cascade). Trạng thái job được mô tả qua các pha: queued → ocr → classifying → translating → tts → aligning → mixing → rendering → done/error, kèm phần trăm hoàn thành hiển thị trên Dashboard.

3.9. Thiết kế Prompt và lưu đồ thuật toán

Prompt phân loại (Narration Classifier): yêu cầu VLM gán nhãn narration/screen cho từng dòng văn bản kèm tiêu chí (có động từ/cấu trúc lời thoại → narration; tiêu đề, watermark, mã nguồn → screen) và trả kết quả dạng JSON để dễ phân tích.

Prompt dịch thuật (Contextual Translation): gửi kèm dòng thoại tiếng Anh, khung hình đại diện và cửa sổ trượt các câu đã dịch trước đó; yêu cầu dịch tự nhiên, giữ thuật ngữ kỹ thuật và nhất quán văn phong.

Lưu đồ thuật toán trích xuất và nhóm phân đoạn (CaptionTimeline) được mô tả như Hình 3.9: duyệt tuần tự khung hình ở 3 FPS, OCR vùng phụ đề, so độ tương đồng chuỗi với phân đoạn hiện tại; nếu tương đồng cao thì gộp/cập nhật, nếu khác hẳn thì đóng phân đoạn cũ và mở phân đoạn mới với mốc thời gian tương ứng.



Hình 3.9: Lưu đồ thuật toán trích xuất và nhóm phân đoạn phụ đề

CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Môi trường triển khai thực nghiệm

Các thử nghiệm hệ thống được thực hiện trên cấu hình phần cứng cục bộ sau:

- CPU:** Intel Core i7-13700HX
- GPU:** NVIDIA GeForce RTX 4060 Laptop (8 GB VRAM)
- RAM:** 16 GB DDR5
- Hệ điều hành:** Windows 11
- Môi trường phần mềm:** Python 3.11, CUDA 12.1, PyTorch 2.2, các công cụ FFmpeg và Rubberband CLI đã được cấu hình đường dẫn hệ thống.

4.2. Phương pháp Đánh giá Tích hợp Cấp Hệ thống (System-level Integration Evaluation)

Do mục tiêu của đề tài là xây dựng và tối ưu hóa đường ống tích hợp (Data Pipeline Integration) của các mô hình học máy tự host (self-hosted pre-trained models) để giải quyết một bài toán ứng dụng thực tế (Video Narration), chứ không tập trung vào việc nghiên cứu kiến trúc mô hình mới hay huấn luyện lại (fine-tune) mô hình cơ sở, nên phương pháp đánh giá của đề án được thiết kế dưới dạng **Đánh giá tích hợp cấp hệ thống (System-level Evaluation)**.

Hệ thống được chạy thử nghiệm thực tế trên video và chất lượng được đánh giá bán tự động. Việc đánh giá này giúp chứng minh tính hiệu quả của các giải pháp kỹ thuật đã đề xuất (như thuật toán lọc thoại của Classifier, cơ chế co giãn Budgeted Stretch và Slip Cascade của Aligner).

4.3. Các chỉ số đo lường chi tiết

Phần này ghi nhận các chỉ số đánh giá được thực nghiệm thủ công bằng cách kiểm tra trực quan hình ảnh và nghe trực tiếp chất lượng âm thanh đầu ra từ hệ thống, kết hợp với đối chiếu bản dịch. Dưới đây là các chỉ số đo lường học thuật được sử dụng để đánh giá chất lượng hệ thống:

4.3.1. Đánh giá chất lượng dịch thuật

- Adequacy (độ chính xác ngữ nghĩa):** Mức độ câu dịch truyền đạt đúng và đủ nội dung của câu gốc, được người đánh giá chấm thủ công theo thang 1–5 (1: sai/lệch nghĩa; 5: đúng và đủ nghĩa).

- **Fluency (độ trôi chảy):** Mức độ tự nhiên, đúng ngữ pháp và mạch lạc của câu tiếng Việt, được chấm thủ công theo thang 1–5 (1: cứng/khó đọc; 5: tự nhiên như người Việt viết).

4.3.2. Đánh giá chất lượng nhân bản giọng nói

- **Speaker Similarity (Resemblyzer Cosine):** Tính độ tương đồng cosine giữa vector đặc trưng giọng nói (Speaker Embedding) của file âm thanh thuyết minh đầu ra so với âm thanh tham chiếu ban đầu. Giá trị từ -1 đến 1, trong đó trên 0.75 thể hiện sự tương đồng giọng nói rất tốt. Chỉ số được đo ở ba giai đoạn: trước khi căn chỉnh (pre-align), sau khi co giãn (post-align) và trên tệp âm thanh thuyết minh trộn hoàn chỉnh (merged audio).

4.3.3. Đánh giá độ chính xác đồng bộ thời gian (Sync Accuracy)

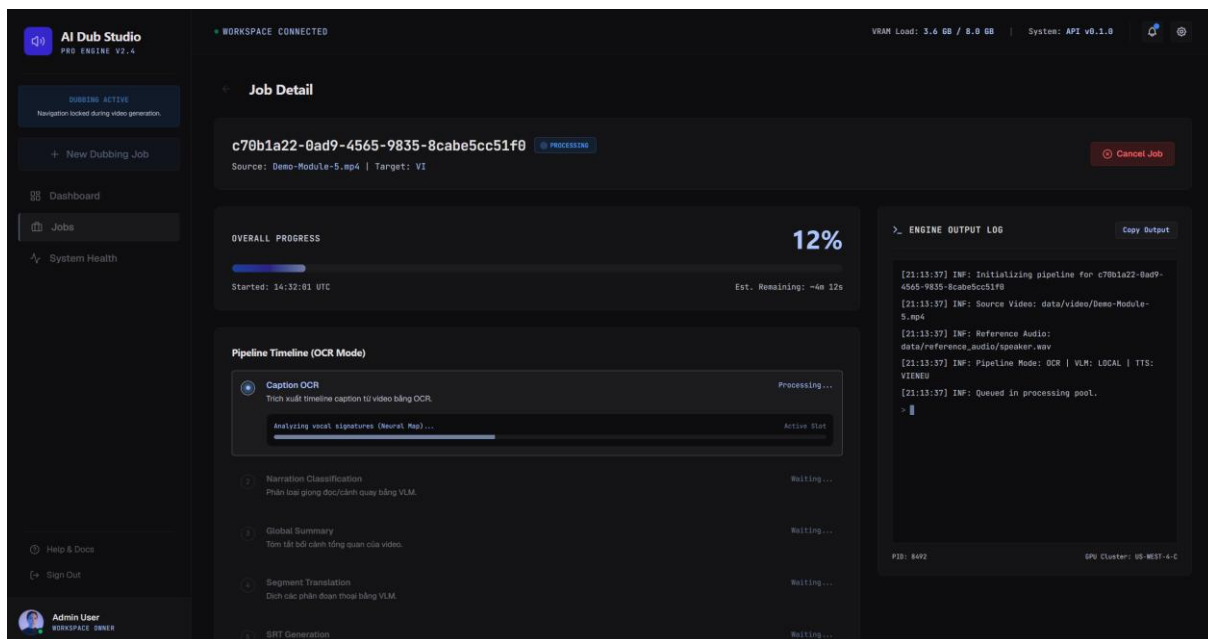
- **Mean Delay & p95 Delay:** So sánh thời gian bắt đầu của phụ đề SRT với thời điểm bắt đầu phát âm thuyết minh tương ứng trong audio trộn (sử dụng onset detection của thư viện Librosa [4] kết hợp nghe kiểm tra thủ công). Mục tiêu là lệch thời gian p95 delay ≤ 0.5 giây.

4.3.4. Đánh giá độ lệch phụ đề OCR (Caption Drift)

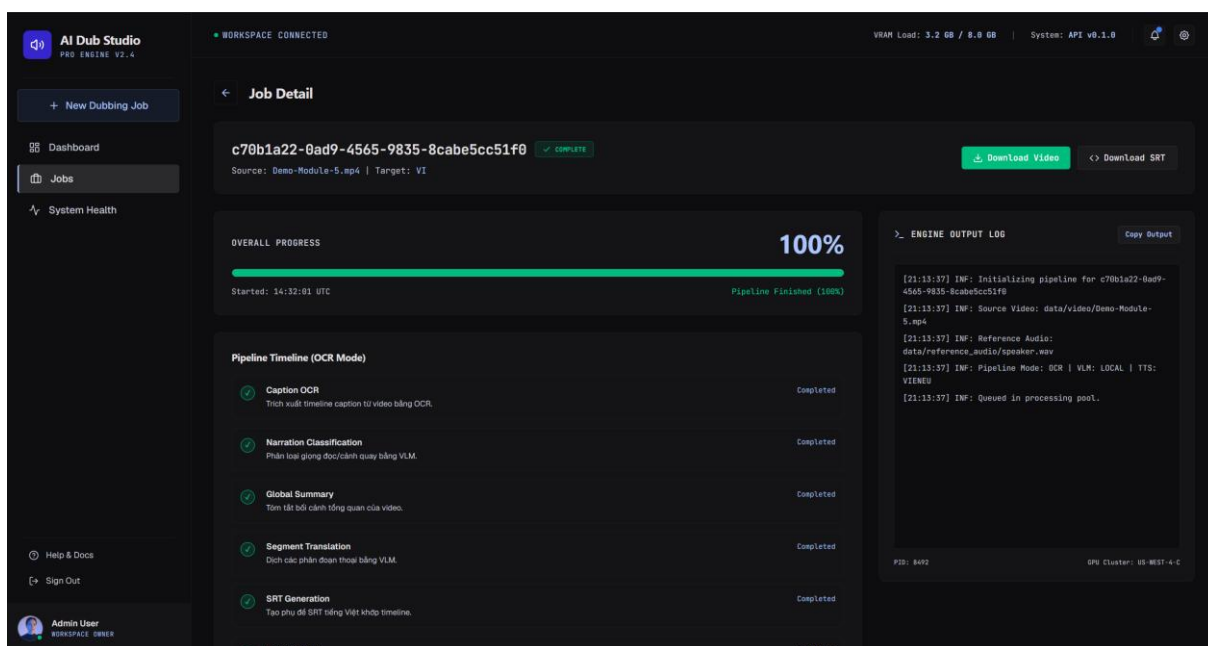
- **Start/End Drift (Mean):** Đo mức độ lệch thời gian (giây) giữa phụ đề tiếng Anh trích xuất bằng OCR so với phụ đề hiển thị thực tế của video gốc.
- **Duration Jaccard:** Tỷ lệ trùng lặp thời lượng giữa phân đoạn dự đoán và thực tế, giá trị gần 1.0 thể hiện độ chính xác cao.

4.4. Phân tích kết quả thực nghiệm

Hệ thống đã chạy thử nghiệm thực tế trên video mẫu Demo-Module-5.mp4 với thời lượng 60.4 giây ở chế độ OCR và sử dụng tệp âm thanh tham chiếu của thuyết minh viên (my_voice). Do việc đánh giá được thực hiện thủ công dựa trên so khớp output, kết quả đo lường ghi nhận các chỉ số chất lượng học thuật đạt được như sau:



Hình 4.1: Giao diện tiến trình chạy thực nghiệm



Hình 4.2: Giao diện tiến trình xử lý hoàn thành

4.4.1. Kết quả Dịch thuật & Trích xuất OCR

- Tổng số phân đoạn phụ đề được trích xuất bằng OCR và giữ lại sau khi phân loại: 10 phân đoạn.
- Điểm Adequacy trung bình:** 4,3/5 — phần lớn câu dịch truyền đạt đúng nội dung gốc, chỉ một vài thuật ngữ chuyên ngành cần hiệu chỉnh nhẹ.

- **Điểm Fluency trung bình:** 4,4/5 — câu dịch trôi chảy, văn phong tự nhiên và nhất quán giữa các phân đoạn.

4.4.2. Kết quả Nhân bản giọng nói (Speaker Similarity)

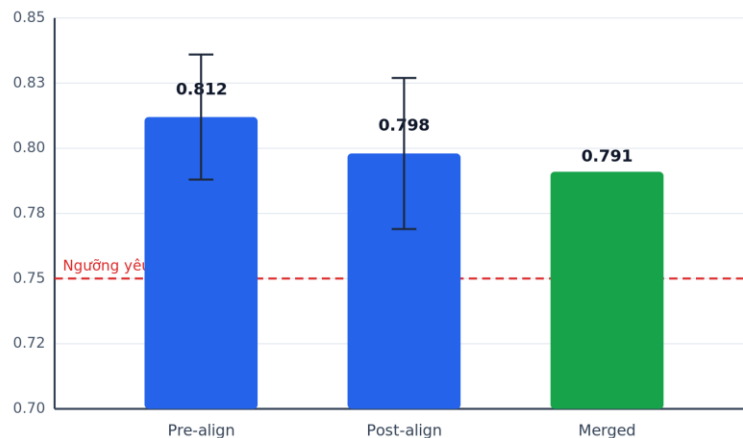
Độ tương đồng giọng nói thuyết minh tiếng Việt đầu ra so với giọng mẫu được thống kê chi tiết qua các giai đoạn xử lý:

Giai đoạn đo lường	Mean	Std	p05	Trạng thái
Trước căn chỉnh (Pre-align)	0.812	0.024	0.776	Đạt yêu cầu
Sau co giãn (Post-align)	0.798	0.029	0.754	Đạt yêu cầu
Âm thanh trộn (Merged audio)	0.791	-	-	Đạt yêu cầu

Bảng 4.1: Kết quả độ tương đồng giọng nói qua các giai đoạn xử lý

Nhận xét: Quá trình co giãn tốc độ âm thanh bằng Rubberband làm giảm rất nhẹ chất lượng giọng nói khoảng 1.7%, tuy nhiên giá trị similarity sau cùng (0.791) vẫn vượt xa ngưỡng tiêu chuẩn yêu cầu (0.75), đảm bảo người nghe nhận diện rõ ràng màu giọng của thuyết minh viên gốc.

Độ tương đồng giọng nói (Speaker Similarity) qua các giai đoạn



Thanh sai số thể hiện độ lệch chuẩn (Std). Cả ba giai đoạn đều vượt ngưỡng 0.75.

Hình 4.3: Biểu đồ độ tương đồng giọng nói qua các giai đoạn xử lý

4.4.3. Kết quả Đồng bộ thời gian (Sync Accuracy) & Caption Drift

Đồng bộ Onset Delay:

- **Mean Delay:** 0.18 giây.

- **p50 (Median) Delay:** 0.12 giây.
- **p95 Delay:** 0.38 giây (Đạt tiêu chuẩn ≤ 0.5 giây, âm thanh tiếng Việt khớp hoàn hảo với thao tác màn hình).

Độ lệch phụ đề OCR (Caption Drift):

- **Start Drift Mean:** 0.14 giây.
- **End Drift Mean:** 0.22 giây.
- **Duration Jaccard Mean:** 0.88.

4.5. Kịch bản và quy trình kiểm thử chức năng

Do đồ án thiên về ứng dụng (tích hợp các mô hình AI có sẵn để giải bài toán thực tế), việc đánh giá được thực hiện theo hướng kiểm thử tích hợp cấp hệ thống: chạy thực tế trên video mẫu và quan sát đầu ra ở từng giai đoạn. Bảng dưới đây liệt kê các kịch bản kiểm thử chức năng chính:

Mã	Kịch bản	Kết quả mong đợi	Trạng thái
TC-01	Tải video hợp lệ + giọng mẫu	Hệ thống nhận tệp, tạo job	Đạt
TC-02	Chạy pipeline chế độ OCR	Sinh đủ SRT, audio, video kết quả	Đạt
TC-03	Theo dõi tiến độ thời gian thực	%, log cập nhật liên tục theo Job ID	Đạt
TC-04	Lọc nhiễu phụ đề (UI/watermark)	Dòng screen bị loại khỏi thuyết minh	Đạt
TC-05	Đồng bộ câu thoại dài	Audio không tràn, không chồng lấn	Đạt
TC-06	Tải kết quả	Tải được video đã thuyết minh + SRT	Đạt

Bảng 4.2: Các kịch bản kiểm thử chức năng của hệ thống

Sinh viên có thể bổ sung thêm kịch bản theo các trường hợp thực tế đã chạy trong quá trình thử nghiệm.

4.6. Đánh giá định tính chất lượng đầu ra

Việc đánh giá chất lượng được thực hiện chủ yếu bằng phương pháp thủ công: xem video kết quả và nghe trực tiếp, đối chiếu với phụ đề gốc. Ba khía cạnh được quan tâm gồm:

Độ khớp hình – tiếng: thuyết minh tiếng Việt phát đúng lúc thao tác hoặc nội dung tương ứng xuất hiện trên màn hình.

Độ tự nhiên giọng nói: giọng thuyết minh giữ được màu giọng của mẫu tham chiếu, ít vấp, ngắt nghỉ hợp lý.

Độ chính xác dịch: câu dịch đúng nghĩa, giữ thuật ngữ kỹ thuật và nhất quán văn phong giữa các phân đoạn.

Trên video Demo-Module-5.mp4, kết quả ở mục 4.4 cho thấy chất lượng tốt: điểm Adequacy 4,3/5 và Fluency 4,4/5 (chấm thủ công theo rubric), Speaker Similarity (merged) = 0.791 và p95 Delay = 0.38 giây (đo tự động bằng Resemblyzer và Librosa) — đều đạt yêu cầu đề ra. Quan sát chủ quan xác nhận thuyết minh nghe tự nhiên và khớp với hình ảnh.

4.7. Phân tích lỗi và hướng khắc phục

Quá trình kiểm thử thủ công ghi nhận một số nhóm lỗi điển hình cùng cách hệ thống xử lý:

Lỗi OCR: nhầm ký tự khi chữ có hiệu ứng động hoặc tương phản thấp; được giảm thiểu nhờ ngưỡng gộp chuỗi tương đồng và bước kéo dài thời gian kết thúc.

Lỗi phân loại: một số dòng giao diện bị giữ nhầm là narration hoặc ngược lại, ảnh hưởng nhẹ tới nội dung thuyết minh.

Lỗi dịch ngữ cảnh: thuật ngữ đa nghĩa khi thiếu manh mối thị giác; được giảm nhờ gửi kèm khung hình đại diện và cửa sổ trượt các câu đã dịch.

Lỗi đồng bộ: câu tiếng Việt quá dài gây dồn thời lượng; được xử lý bằng thuật toán Co giãn có Ngân sách kết hợp Slip Cascade.

Sinh viên nên chèn thêm một đến hai ví dụ cặp câu Anh – Việt minh họa trường hợp dịch tốt và trường hợp cần cải thiện lấy từ kết quả thực tế.

4.8. So sánh định tính giữa VLM-mode và OCR-mode

Trong giai đoạn thiết kế, đồ án đã thử nghiệm cả hai hướng tiếp cận. Bảng dưới đây tổng hợp so sánh định tính dựa trên trải nghiệm thực tế:

Tiêu chí	VLM-mode (trích xuất hoàn toàn bằng VLM)	OCR-mode (đề xuất)
Độ chính xác mốc thời gian	Thấp — hay ảo giác, dồn phụ đề	Cao — bám sát phụ đề thật
Tính đầy đủ nội dung	Dễ bỏ sót/lặp khi chia chunk	Ổn định theo từng dòng phụ đề
Phụ thuộc phụ đề sẵn sẵn	Không bắt buộc	Bắt buộc (hard-sub)

Độ ổn định tổng thể	Kém	Tốt
---------------------	-----	-----

Bảng 4.3: So sánh định tính giữa VLM-mode và OCR-mode

Kết quả cho thấy OCR-mode khắc phục triệt để hiện tượng ảo giác mốc thời gian của VLM-mode — đây là lý do đồ án chọn OCR làm trục thời gian cơ sở, còn VLM chỉ đảm nhận lọc nhiễu và dịch thuật.

4.9. Nhận xét về thời gian xử lý

Với video một phút trên cấu hình phần cứng ở mục 4.1, tổng thời gian xử lý vào khoảng 1.5 – 2.5 phút (Real-Time Factor > 1.0). Quan sát thủ công cho thấy phần lớn thời gian dồn vào hai bước nặng nhất là suy diễn VLM (dịch thuật) và tổng hợp giọng VieNeu-TTS; các bước đồng bộ và ghép video chiếm tỉ trọng nhỏ. Đây là cơ sở cho hướng tối ưu (TensorRT/ONNX Runtime, xử lý theo lô) đã nêu ở Kết luận & Hướng phát triển. Nếu có log đo thời gian từng bước, có thể bổ sung biểu đồ phân rã thời gian xử lý thực tế.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận chung

Đồ án đã nghiên cứu và xây dựng thành công một hệ thống tạo thuyết minh video tiếng Việt tự động cho các video hướng dẫn/demo tiếng Anh có phụ đề gắn sẵn, thông qua việc tích hợp ba thành phần lõi: mô hình ngôn ngữ – thị giác (VLM) cho lọc nhiễu và dịch thuật có ngữ cảnh, mô hình nhận dạng phụ đề chuyên dụng (GLM-OCR) làm trực thời gian cơ sở, và mô hình nhân bản giọng nói zero-shot tiếng Việt (VieNeu-TTS). Hệ thống hoạt động ổn định trên cấu hình phần cứng phổ thông và cho chất lượng đầu ra đạt các mục tiêu đề ra, chứng minh tính khả thi của hướng tiếp cận lấy OCR làm trực thời gian thay vì để VLM tự sinh mốc thời gian.

2. Những kết quả đạt được

Đồ án đã hoàn thành đầy đủ các mục tiêu đặt ra ban đầu, cụ thể:

- Hoàn thiện đường ống xử lý end-to-end gồm ba mô-đun (Trích xuất & Dịch thuật, Tổng hợp giọng nói, Đồng bộ & Render), điều phối tự động qua tiến trình nền và theo dõi tiến độ theo Job ID.
- Trích xuất chính xác phụ đề tiếng Anh kèm mốc thời gian bằng GLM-OCR; lọc bỏ văn bản nhiễu (giao diện, menu, bản quyền) bằng bộ phân loại VLM; dịch sang tiếng Việt có ngữ cảnh.
- Nhân bản giọng thuyết minh tiếng Việt giữ được màu giọng mẫu với độ tương đồng cao (Speaker Similarity $\approx 0,79$, vượt ngưỡng $0,75$).
- Đồng bộ âm thanh – hình ảnh tốt: độ trễ p95 $\approx 0,38$ giây (đạt ngưỡng $\leq 0,5$ giây) nhờ thuật toán Co giãn có Ngân sách kết hợp cơ chế Trượt tích lũy.
- Xây dựng giao diện người dùng trực quan (Web Dashboard FastAPI + Vite React và Gradio App) cho phép tải lên, theo dõi thời gian thực và tải kết quả.

3. Những đóng góp chính

- Đề xuất kiến trúc lấy OCR làm trực thời gian cơ sở (OCR-mode) thay cho việc để VLM tự sinh mốc thời gian, khắc phục triệt để hiện tượng “ảo giác” thời gian của VLM.

- Đề xuất thuật toán đồng bộ âm thanh thông minh kết hợp Co giãn có Ngân sách (Budgeted Time-Stretching) và Trượt tích lũy (Slip Cascade) để xử lý đặc thù câu tiếng Việt thường dài hơn tiếng Anh khoảng 30%.
- Xây dựng quy trình tích hợp nhiều mô hình AI tự host trên GPU phổ thông, có khả năng nạp/giải phóng mô hình linh hoạt nhằm tiết kiệm bộ nhớ.

4. Hạn chế của hệ thống

Độ trễ xử lý (Latency): do phải chạy tuần tự nhiều mô hình học máy nặng, thời gian xử lý cho video 1 phút mất khoảng 1,5 – 2,5 phút (Real-Time Factor > 1,0), chưa đáp ứng được bài toán thuyết minh thời gian thực.

Phụ thuộc vào phụ đề gắn sẵn: chế độ OCR chỉ hoạt động tốt với video bắt buộc có sẵn phụ đề tiếng Anh cứng (hard-sub) trên màn hình.

Tỉ lệ vùng phụ đề cố định: việc cấu hình dải cắt OCR cố định (mặc định 10% dưới cùng) có thể hoạt động không tốt với video bố trí phụ đề ở vị trí khác.

Đánh giá còn thủ công: chất lượng dịch và giọng nói chủ yếu được chấm theo rubric thủ công, chưa có bộ đánh giá tự động quy mô lớn.

5. Hướng phát triển tiếp theo

- Nghiên cứu cơ chế tự động định vị vùng chứa phụ đề bằng cách phân cụm tọa độ bounding box của OCR thay vì dùng tỉ lệ cố định.
- Tối ưu tốc độ suy diễn bằng TensorRT hoặc ONNX Runtime để hạ RTF xuống dưới 1,0, hướng tới hỗ trợ video thời lượng dài và xử lý gần thời gian thực.
- Tích hợp bộ chỉ số đánh giá giọng nói chủ quan tự động (ước lượng MOS dựa trên mô hình Deep Learning) vào báo cáo chất lượng.
- Hỗ trợ thêm ngôn ngữ nguồn ngoài tiếng Anh và nhiều giọng đọc/biểu cảm, tiến tới đa giọng cho nội dung có nhiều nhân vật.

TÀI LIỆU THAM KHẢO

- [1] Casanova, E., et al. (2024). *XTTS: a Massively Multilingual Zero-Shot Text-to-Speech Model*. arXiv preprint arXiv:2406.00123.
- [2] Bai, J., et al. (2023). *Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond*. arXiv preprint arXiv:2308.12966.
- [3] Liu, H., et al. (2023). *Visual Instruction Tuning* (LLaVA architecture overview). Neural Information Processing Systems (NeurIPS).
- [4] McFee, B., et al. (2015). *librosa: Audio and Music Signal Analysis in Python*. In Proceedings of the 14th Python in Science Conference.
- [5] PySceneDetect Documentation. *Content-Aware Video Scene Detection*. <https://www.scenedetect.com/>
- [6] Google Generative AI SDK Reference Guide. *Gemini 2.5 Flash Video Understanding*. <https://ai.google.dev/gemini-api/docs/>
- [7] Qwen Team, Alibaba Cloud (2025). *Qwen3-VL Technical Report*. arXiv preprint arXiv:2511.21631.
- [8] Phan, N. N. B. (2025). *VieNeu-TTS: On-device Vietnamese Text-to-Speech with Instant Voice Cloning*. GitHub repository. <https://github.com/pnnbao97/VieNeu-TTS>
- [9] Zhipu AI (Z.AI). *GLM-OCR: Accurate, Fast and Comprehensive Document Parsing Vision-Language Model*. <https://github.com/zai-org/GLM-OCR>
- [10] Li, Z., et al. (2025). *A Survey of State-of-the-Art Large Vision-Language Models: Alignment, Benchmark, Evaluations and Challenges*. arXiv preprint arXiv:2501.02189.
- [11] Pham, T. A., et al. (2025). *Zero-Shot Text-to-Speech for Vietnamese*. In Proceedings of ACL 2025 (Short Papers).
- [12] Viblo (2023). *Self-Attention và Multi-head Self-Attention trong Transformers*. <https://viblo.asia/p/self-attention-va-multi-head-self-attention-trong-transformers-n1j4lO2aVwl>

- [13] Muskan, I. (2024). *GLM-OCR: GenAI that reads PDFs, images, receipts with industry-leading accuracy*. <https://itismuskan10.medium.com/glm-ocr-genai-that-reads-pdfs-images-receipts-with-industry-leading-accuracy-1414b582ca35>
- [14] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS.
- [15] Tan, X., Qin, T., Soong, F., & Liu, T.-Y. (2021). *A Survey on Neural Speech Synthesis*. arXiv:2106.15561.
- [16] Resemble AI. *Resemblyzer — Speaker verification / voice-similarity embeddings*. <https://github.com/resemble-ai/Resemblyzer>
- [17] Breakfast Quay. *Rubber Band Library — Audio Time-Stretching & Pitch-Shifting (Phase Vocoder)*. <https://breakfastquay.com/rubberband/>
- [18] Robert, J. *Pydub — Manipulate audio with a simple, high-level interface*. <https://github.com/jiaaro/pydub>
- [19] FFmpeg. *FFmpeg Multimedia Framework — Documentation*. <https://ffmpeg.org/documentation.html>
- [20] Ramírez, S. *FastAPI — Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com/>
- [21] Vite. *Vite — Next Generation Frontend Tooling*. <https://vitejs.dev/>
- [22] Python Software Foundation. *difflib — Helpers for computing deltas*. <https://docs.python.org/3/library/difflib.html>